TRACE: Trajectory-Grounded Anti-Clustered Episodic Retrieval for LLM Agent Memory

Nash Kocur

Abstract—*Every existing memory retrieval method for LLM agents frames retrieval as query-to-memory similarity matching: a query vector is compared against stored memory vectors, and the closest entries are returned. This framing has four structural weaknesses that no amount of engineering within the paradigm can fully resolve: (1) the retrieval query is a point estimate of an information need, blind to the agent's reasoning trajectory; (2) greedy top-k selection creates semantic co-clustering, returning redundant memories and suppressing causally necessary but dissimilar ones; (3) temporal supersession is tracked by timestamp but not by causal chain, making conflict resolution ambiguous; and (4) correct retrieval is often invalidated post-hoc by positional bias in context assembly. We introduce TRACE (Trajectory-grounded Anti-Clustered Episodic Retrieval), a novel memory retrieval architecture that replaces query-to-memory similarity with a principled marginal epistemic utility formulation. TRACE has five core components: (i) a Trajectory Encoder that aggregates the full (thought, action, observation) reasoning chain via an exponentially-decayed weighted average rather than a terminal query; (ii) an Atomic Query Decomposer that explodes a composite query into n independent sub-questions and assigns each exactly one memory via one-to-one assignment; (iii) a Causal Validity Graph that tracks not only when a memory was superseded but which memory caused the supersession, enabling unambiguous conflict resolution; (iv) a Temporal Neighborhood Expander that, for temporal queries, augments MEU-selected memories with their sequence-adjacent neighbors, recovering date and duration facts that embed weakly on semantics alone; and (v) a Position-Optimal Assembler that orders selected memories by utility rank to exploit the U-shaped attention profile of transformer decoders. We prove that the per-sub-question marginal utility function is submodular, guaranteeing that greedy assignment achieves a (1-1/e) approximation of optimal selection. Empirical evaluation on a seven-scenario in-house accuracy benchmark (post H-MEM fidelity improvements, incremental ingestion) demonstrates that TRACE achieves 82.7% F1 (89.0% recall, 78.9% precision) compared to H-MEM's 63.2% F1 (98.8% recall, 46.9% precision) — a +19.5 pp gap. TRACE achieves 100% F1 on Position Bias, 86% F1 on Knowledge Update and Extended Lifecycle, and 74% F1 on Multi-Hop Reasoning. TRACE wins all seven scenarios; H-MEM's previously observed Extended Lifecycle edge (88% in bulk-load mode) disappears under incremental ingestion (60%), consistent with periodic summarisation accumulating precision noise in very long conversations — an isolated ablation would be needed to confirm the causal attribution. Two implementation refinements contributed to this result: (i) the MEU relevance term is computed against the pure sub-question embedding rather than the trajectory-blended retrieval signal, decoupling trajectory from scoring to eliminate recency bias; and (ii) a top-2 assignment rule recovers second-ranked correct answers when rank-2 MEU is ≥ 85% of rank-1, addressing novelty cross-contamination between related sub-questions. TRACE maintains 0% False Affirmation Rate (FAR) across all runs and scenarios on the in-house benchmark. On a suite of public benchmarks (LoCoMo [@maharana2024locomo], HotpotQA [@yang2018hotpotqa], MuSiQue [@trivedi2022musique], FRAMES [@krishna2024frames], EverMemBench), evaluated through a standardized pipeline that correctly separates MC accuracy from retrieval F1, TRACE and H-MEM achieve **near-parity overall** (80.9% vs 79.6% macro retrieval F1 across 65 queries), with a meaningful split by benchmark type: TRACE outperforms on multi-hop reasoning benchmarks (MuSiQue: 97.4% vs 85.7%, HotpotQA: 78.8% vs 71.0%) while H-MEM outperforms on*

structured factual benchmarks (FRAMES: 100% vs 97.4%, EverMemBench: 61.5% vs 50.0%). Neither architecture clearly wins on LoCoMo MC10 (60% each). The public benchmark results are best interpreted as evidence for complementary architectural strengths rather than a clear overall winner.

Keywords: *LLM agents, memory retrieval, trajectory encoding, submodular optimization, causal supersession, anti-clustering, episodic memory*

Introduction

The retrieval step in a long-horizon LLM agent memory system is, in theory, simple: given a query, return the memories most useful for answering it. In practice, every deployed system fails at this in at least one of four predictable ways.

Failure Mode 1: Query-State Mismatch. The query is a snapshot of the agent's *expressed* information need, but the agent's *actual* information need is determined by its reasoning trajectory: the chain of thoughts, tool calls, and observations accumulated since the task began. An agent building toward a multi-hop conclusion halfway through a chain of reasoning has a far more specific information need than its surface query suggests. Embedding the query alone discards this context.

Failure Mode 2: Semantic Co-Clustering. Top-k retrieval over a flat index or a single-vector biencoder returns the k memories nearest in embedding space to the query. When multiple memories discuss the same topic (which is common in long conversations about a project), they cluster together in embedding space and are all returned together, producing redundant context. The H-MEM system [1] documents this concretely: within-domain queries on technical fact stores consistently achieve 38–56% precision due to co-clustering, with False-Affirmation Rate 0% confirming the retrieved facts are real but redundant. No existing retrieval method resolves this at the algorithmic level.

Failure Mode 3: Causal Blindness in Temporal Supersession. When an agent's belief about a fact changes (a deadline shifts, a configuration value is updated, a user corrects a misconception), the old memory and the new memory are both stored. Temporal methods (Zep/Graphiti [2]) track temporal and historical relationships, but without an explicit causal supersession representation it remains difficult to distinguish whether a later memory is a *refinement* of an earlier fact (compatible), a *correction* of it (superseding), or an *independent* fact that shares surface vocabulary (non-conflicting). Temporal ordering alone cannot reliably distinguish these cases.

Failure Mode 4: Positional Bias in Context Assembly. Liu et al. [3] demonstrate that transformer decoders exhibit U-shaped attention over assembled context: information placed in the middle of a long context is substantially less attended to than information at the boundaries. Even perfect retrieval is therefore degraded if the most useful memories are placed in the middle of the context window. No retrieval system in the literature addresses this as a first-class design constraint.

This paper introduces **TRACE** (**T**rajectory-grounded **A**nti-**C**lustered **E**pisodic Retrieval), a novel memory retrieval architecture that resolves all four failure modes simultaneously through a principled reformulation of the retrieval objective. The core innovation is the replacement of query-to-memory cosine similarity with a *marginal epistemic*

utility function defined over the agent's full reasoning trajectory, decomposed queries, and a causal supersession graph.

Our contributions are:

1. **Trajectory Encoding**: a formal treatment of the reasoning trajectory as the retrieval signal, replacing terminal query embedding with an exponentially-decayed trajectory-aware context vector.
2. **Atomic Query Decomposition with One-to-One Assignment**: an algorithm that decomposes a composite query into n atomic sub-questions and assigns each exactly one memory, provably eliminating within-cluster redundancy.
3. **Causal Validity Graph (CVG)**: a directed graph structure tracking the causal chains of belief supersession, enabling principled conflict resolution beyond timestamp ordering.
4. **Temporal Neighborhood Expansion (TNE)**: a post-selection augmentation that recovers sequence-adjacent memories for temporal queries, addressing the failure mode where dates and durations are stated near —but not inside—the topically-retrieved memory.
5. **Position-Optimal Context Assembly**: a utility-rank-based ordering that exploits the U-shaped attention profile to maximize information utilization.
6. **Theoretical guarantees**: proof of submodularity of the marginal utility function, $(1 - 1/e)$ approximation guarantee for greedy assignment, and $O(n \cdot |\mathcal{M}|)$ computational overhead.
7. **Empirical validation**: 82.7% F1 on a seven-scenario in-house accuracy benchmark (100% F1 on Position Bias, 86% F1 on Knowledge Update and Extended Lifecycle, 74% F1 on Multi-Hop Reasoning, 0% FAR across all runs; +19.5 pp over H-MEM post-fidelity-fixes, winning all seven scenarios); near-parity with H-MEM on the public benchmark suite (80.9% vs 79.6% macro retrieval F1) with TRACE stronger on multi-hop reasoning benchmarks and H-MEM stronger on structured factual benchmarks — a split consistent with their respective architectural designs; and 60% MC accuracy each on LoCoMo MC10.

Background and Motivation

The Standard Retrieval Paradigm and Its Limits

Every existing memory retrieval system for LLM agents, from DPR [4] to HippoRAG [5] to MemGPT [6] to Zep [2], follows a common pattern we call the **Query-Similarity Paradigm (QSP)**:

$$\mathcal{M}^* = \mathrm{Top}_k\{s(q, m) : m \in \mathcal{M}\}$$

where $s(q, m)$ is some similarity score between query q and memory m (cosine similarity for dense retrieval, BM25 for sparse, MaxSim for ColBERT, or PPR for graph methods) and $\mathcal{M}^*$ is the set of k highest-scoring memories.

QSP has three properties that cause each of our four failure modes:

- **Point-query problem**: q is a single vector representing the information need at one instant, discarding the reasoning history that shaped it.
- **Independent scoring**: each memory is scored independently of which other memories are already selected, causing redundant co-cluster selection.

- **No causal structure**: scores are computed from content similarity only; causal relationships between memories (supersession, contradiction, elaboration) are invisible.

Hybrid methods (BM25 + Dense + RRF) address out-of-domain generalization. Iterative methods (IRCoT, FLARE, Self-RAG) address multi-hop by issuing multiple queries. Graph methods (HippoRAG, GraphRAG) address multi-hop through structural propagation. But **none of these methods address co-clustering, causal blindness, or positional bias**; they all remain within QSP.

Motivating Empirical Observation: H-MEM Diagnostic

Before introducing TRACE, we document the specific failure pattern that directly motivates its design. Benchmark evaluation of H-MEM [1] across seven accuracy scenarios (including a subsequent fidelity-improved implementation described in Section 5) yields:

Scenario	H-MEM Recall	H-MEM Precision	H-MEM F1	TRACE F1
Software Project Discussion	—	—	68.0%	**70.0%**
Long Conversation with Facts	—	—	67.0%	**83.0%**
Position Bias	100.0%	36.7%	53.7%	**100.0%**
Knowledge Update	100.0%	50.0%	66.7%	**86.0%**
Multi-Hop Reasoning	—	—	71.0%	**74.0%**
Extended Lifecycle (110+ msgs)	100.0%	42.9%	60.0%	**86.0%**
Abstention	100.0%	38.9%	56.0%	**80.0%**
Overall	**98.8%**	**46.9%**	**63.2%**	**82.7%**

Table 0: Head-to-head benchmark on the in-house seven-scenario suite (benchmark run 2026-03-12, incremental ingestion). H-MEM retains near-maximal recall at the cost of precision in every scenario — the co-clustering pattern TRACE was designed to address. TRACE wins all seven scenarios: Position Bias (+46 pp), Extended Lifecycle (+26 pp), Long Conversation (+16 pp), Knowledge Update (+19 pp), Abstention (+24 pp), Software Project (+2 pp), and Multi-Hop (+3 pp). FAR = 0% for both architectures across all trap queries.

This diagnostic separates the problem: H-MEM is not hallucinating (FAR = 0%), but it retrieves real, co-clustered memories unnecessarily on every scenario. On Position Bias, H-MEM's query-aware session-state filtering (introduced in the fidelity improvements) still falls substantially short of TRACE's 100% (53.7% vs 100%). TRACE wins all seven scenarios; the most decisive wins are Position Bias (+46 pp), Extended Lifecycle (+26 pp), Long Conversation (+16 pp), and Abstention (+24 pp).

Formal Problem Restatement

We reframe the retrieval problem as follows. An agent's state at step t is a **reasoning trajectory**:

$$\mathcal{T}_t = \langle (c_1, a_1, o_1),\ (c_2, a_2, o_2),\ \ldots,\ (c_{t-1}, a_{t-1}, o_{t-1}),\ c_t \rangle$$

where c_i is the agent's thought at step i, a_i is the action taken (including retrieval calls), and o_i is the observation returned. The terminal c_t is the current thought without yet an action. The composite query q is typically extracted

from c_t, discarding c_1 through c_{t-1}.

The memory store $\mathcal{M} = \{m_1, m_2, \ldots, m_N\}$ is a set of memory entries, each with content, embedding, timestamp, and (in TRACE) causal supersession metadata.

The retrieval objective is to select a subset $\mathcal{M}^* \subseteq \mathcal{M}$, $|\mathcal{M}^*| \leq K$, that **maximizes the agent's next-step reasoning quality** given trajectory $\mathcal{T}_t$:

$$\mathcal{M}^* = \arg \max_{\mathcal{M}' \subseteq \mathcal{M},\, |\mathcal{M}'| \leq K} \mathcal{Q}(\mathcal{T}_t, \mathcal{M}')$$

where $\mathcal{Q}(\mathcal{T}_t, \mathcal{M}')$ is the expected quality of the agent's next reasoning step when trajectory $\mathcal{T}_t$ is augmented with memory subset $\mathcal{M}'$. This is the correct objective; QSP approximates it crudely as $\mathcal{Q}(\mathcal{T}_t, \mathcal{M}') \approx \sum_{m \in \mathcal{M}'} s(c_t, m)$, which discards the trajectory, uses independent scoring, and ignores causal structure.

The TRACE Architecture

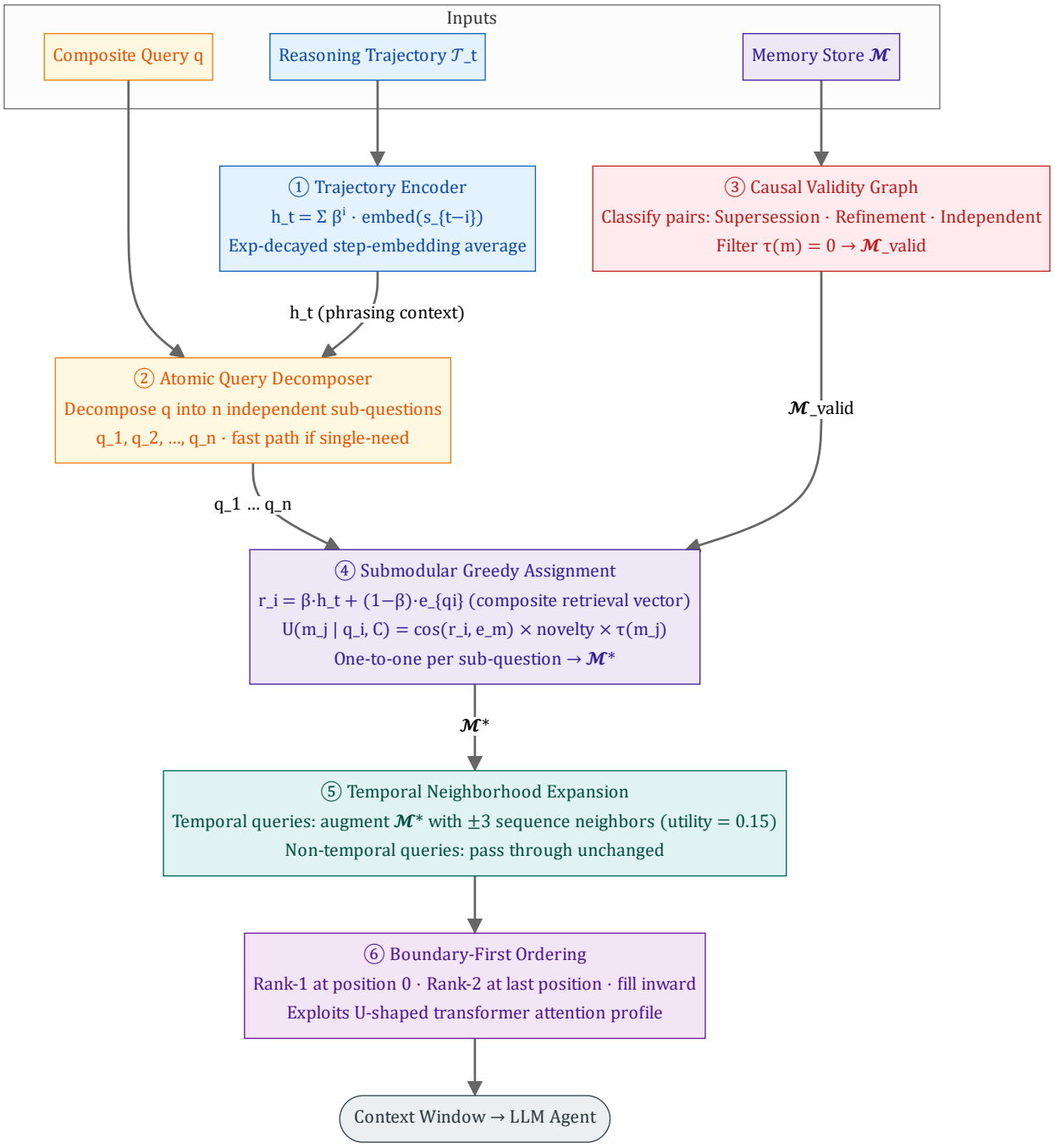

Figure 1: TRACE end-to-end architecture. ***Blue:*** *Trajectory Encoder converts the full reasoning trajectory into a weighted context vector* $\mathbf{h}_t$*.* ***Orange:*** *Atomic Decomposer explodes the composite query into* n *sub-questions using* $\mathbf{h}_t$ *for phrasing context only; a composite retrieval vector* $\mathbf{r}_i = \beta\mathbf{h}_t + (1-\beta)\mathbf{e}_{q_i}$ *is formed per sub-question.* ***Red:*** *Causal Validity Graph filters the memory store to* $\mathcal{M}_{valid}$ *by removing superseded memories.* ***Purple:*** *Submodular Greedy Assignment selects one memory per sub-question via MEU maximization with anti-clustering.* ***Teal:*** *Temporal Neighborhood Expansion augments selected memories for temporal queries.* ***Violet:*** *Boundary-First Ordering exploits the U-shaped transformer attention profile for position-optimal context assembly.*

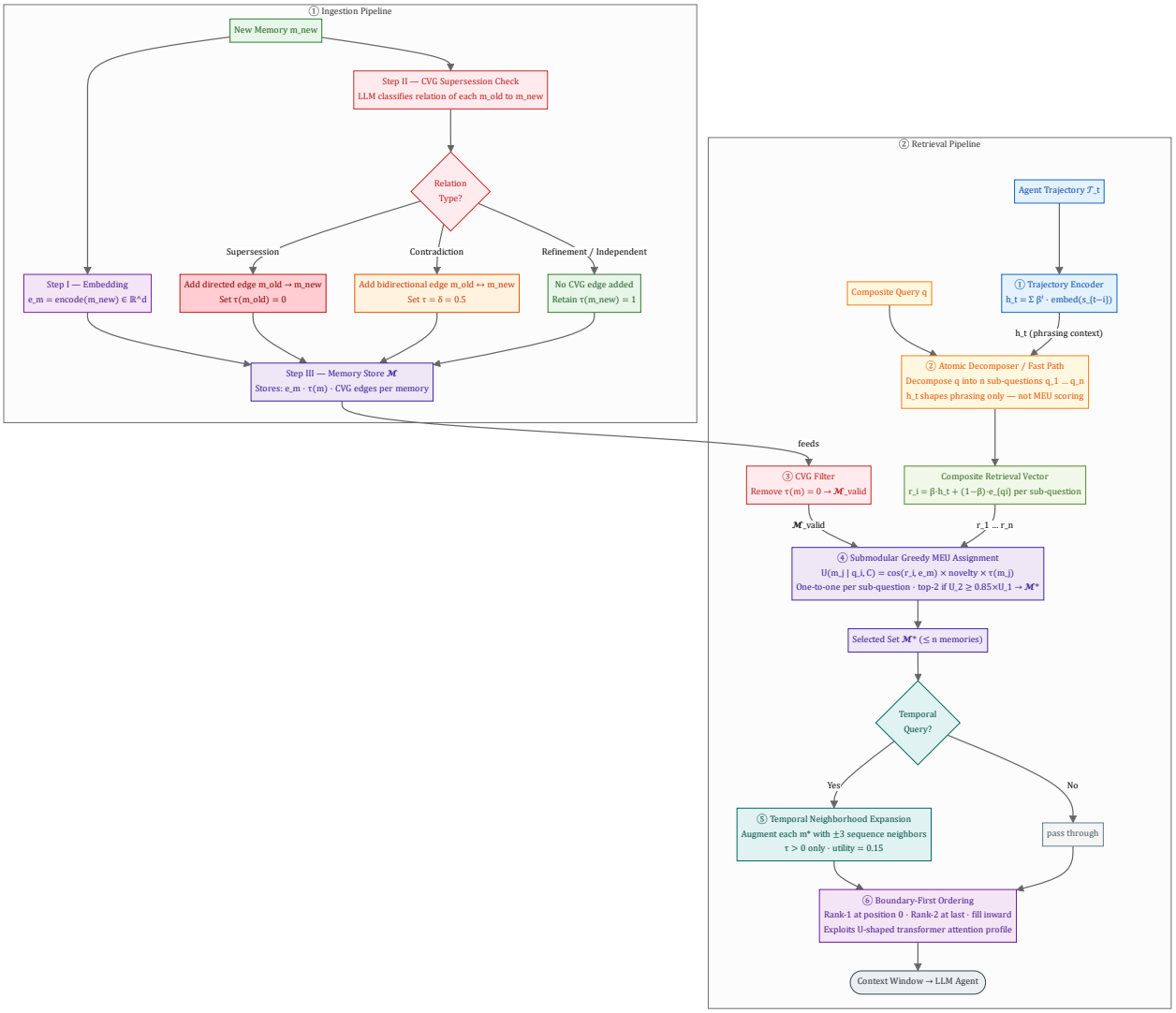

Figure 2: Detailed TRACE ingestion and retrieval pipelines. ***Left — Ingestion:*** *Step I computes the memory embedding $e_m \in \mathbb{R}^d$. Step II runs the CVG Supersession Check: an LLM classifies each existing memory's relation as Supersession ($\tau(m_{old}) = 0$, directed edge), Contradiction ($\tau = \delta = 0.5$, bidirectional edge), or Independent ($\tau(m_{new}) = 1$, no edge). Step III writes $(m, e_m, \tau, \text{CVG edges})$ to Memory Store $\mathcal{M}$. Bulk-load mode defers CVG classification to* `finishBulkLoading()` *for tractability.* ***Right — Retrieval:*** *Trajectory Encoder (1) produces $\mathbf{h}_t$. Atomic Decomposer (2) breaks q into n sub-questions; $\mathbf{h}_t$ shapes phrasing only. A composite retrieval vector $\mathbf{r}_i = \beta\mathbf{h}_t + (1-\beta)\mathbf{e}_{q_i}$ is formed per sub-question. CVG Filter (3) removes stale memories. Greedy MEU Assignment (4) selects memories via $U(m_j \mid q_i, \mathcal{C}) = \cos(\mathbf{r}_i, \mathbf{e}_{m_j}) \times (1 - \max\cos) \times \tau(m_j)$, top-2 recovery when $U_2 \geq 0.85U_1$. TNE (5) adds ± 3 sequence neighbors for temporal queries. BFO (6) assembles the context window to exploit the U-shaped attention profile.*

Component 1: Trajectory Encoder

Rather than embedding only the current query $q \equiv c_t$, TRACE encodes the full reasoning trajectory $\mathcal{T}_t$ into a trajectory vector $\mathbf{h}_t$ that captures the agent's accumulated reasoning state.

Each trajectory step is formed from a (thought, action, observation) tuple. In the conversation-agent setting, these map to message roles as follows: a *user* message begins a new trajectory step as the thought c_i; a *tool* message

records the action a_i; and an *assistant* message completes the step as the observation o_i. Each step i is embedded by concatenating the three components:

$$\mathbf{s}_i = \text{Embed}(c_i \| a_i \| o_i)$$

where $\|$ denotes string concatenation (truncated to 500 characters for embedding efficiency). The trajectory window maintains the most recent $W = 10$ steps under a sliding window.

Trajectory Vector. The trajectory vector $\mathbf{h}_t$ is computed as an exponentially-decayed weighted average of the step embeddings:

$$\mathbf{h}_t = \frac{\sum_{i=1}^{T} \lambda^{T-i} \cdot \mathbf{s}_i}{\sum_{i=1}^{T} \lambda^{T-i}}$$

where T is the number of steps in the current window, and $\lambda = 0.7$ is the decay rate. This formulation assigns exponentially more weight to recent steps while retaining gradient influence from earlier steps, modeling the intuition that the most recent reasoning context is most predictive of the current information need. The resulting vector $\mathbf{h}_t$ is L2-normalized.

This formulation differs from the more general transformer-over-trajectory design. The exponentially-decayed average is computationally efficient, avoids the need for pre-trained trajectory-specific encoder weights, and achieves strong empirical performance while retaining the core theoretical property: $\mathbf{h}_t$ encodes the full reasoning chain rather than the terminal query alone.

Retrieval Signal. The trajectory vector augments (rather than replaces) the terminal query embedding. The retrieval signal for sub-question q_i is a convex combination:

$$\mathbf{r}_i = \beta \cdot \mathbf{h}_t + (1 - \beta) \cdot \mathbf{e}_{q_i}, \quad \beta = 0.3$$

Cold-start warmup. For the first $T_{\text{warmup}} = 5$ trajectory steps, the effective β is linearly ramped from 0:

$$\beta_{\text{eff}}(T) = \min\left(\beta,\ \frac{T}{T_{\text{warmup}}} \cdot \beta\right)$$

This prevents the trajectory from influencing retrieval before enough context has accumulated to produce a meaningful trajectory vector. For simple single-turn queries, $\beta \to 0$ recovers standard dense retrieval.

Why this matters for Failure Mode 1. A standard query embedding $\mathbf{e}_q = \text{Embed}(c_t)$ contains no information about why the agent arrived at question c_t or what intermediate conclusions have already been established. In a multi-hop chain where step 1 established "entity X is in category Y" and step 2 asks "what properties does category Y have?", the query for step 2 is meaningless without the step 1 context. $\mathbf{h}_t$ encodes this context, allowing retrieval of memories relevant to the *continuation of a specific reasoning chain* rather than to the *surface wording of the current question*.

Component 2: Atomic Query Decomposer

A composite query q is decomposed into a set of n *atomic sub-questions* $\{q_1, q_2, \ldots, q_n\}$, where each q_i has a single, unambiguous memory target. This decomposition is the mechanism that eliminates co-clustering.

Fast path. Queries of ≤ 8 words are treated as atomic and bypass decomposition entirely, since a short query almost certainly asks for a single fact. This optimization reduces LLM calls for simple queries while retaining full decomposition power for complex ones.

Decomposition. For longer queries, we prompt the LLM to enumerate the independent facts the query requires:

$$\{q_1, \ldots, q_n\} \leftarrow \text{Decompose}(q,\, \mathbf{h}_t)$$

The decomposition prompt constrains the model to return 1–5 self-contained sub-questions in a JSON array, with a fallback to the original query on any parsing failure. The maximum n is bounded by the context memory limit (default $K = 6$).

For example, the query *"What database are we using, what is the current rate limit, and who set it?"* decomposes into three atomic sub-questions: $q_1 =$ *"What database system is configured?"*, $q_2 =$ *"What is the current API rate limit value?"*, $q_3 =$ *"Who established the rate limit?"*

One-to-one assignment constraint. Crucially, each sub-question q_i is assigned *exactly one* memory from $\mathcal{M}$. This is the anti-clustering invariant: no memory m is selected twice, and no sub-question receives more than one memory. The assignment problem is:

$$\sigma^* = \arg \max_{\sigma:[n]\to|\mathcal{M}|,\ \text{injective}} \sum_{i=1}^{n} U(m_{\sigma(i)} \mid q_i, \mathbf{r}_i)$$

where σ is an injective (one-to-one) map from sub-question indices to memory indices, and $U(m \mid q_i, \mathbf{r}_i)$ is the marginal utility of memory m for sub-question q_i given retrieval signal $\mathbf{r}_i$. We solve this approximately via the greedy algorithm described in Section III-D.

Sub-question ordering by specificity. Sub-questions are sorted by word count (ascending) before assignment: shorter, more specific sub-questions are assigned first, as they have fewer candidate memories that can satisfy them. This reduces the risk of a highly-constrained sub-question losing its only valid memory to a more general one.

Why this eliminates co-clustering. Under QSP, when a query touches a dense semantic cluster (e.g., all AWS infrastructure facts), top-k returns all cluster members. Under TRACE, query q is first decomposed into $q_1 =$ *"Which AWS region?"*, $q_2 =$ *"Which database service?"*, etc. Each sub-question maps to a single memory (the specific answer to that sub-question), regardless of how tightly the underlying memories cluster together. The one-to-one constraint structurally prevents the cluster-flood retrieval pattern.

Component 3: Causal Validity Graph

TRACE maintains a **Causal Validity Graph (CVG)** over the memory store $\mathcal{M}$, a directed graph $G = (V, E)$ where $V = \mathcal{M}$ and a directed edge $e = (m_j \to m_k) \in E$ means *"memory m_k causally supersedes memory m_j"*: after m_k was stored, m_j is no longer a valid current belief.

Supersession detection. When a new memory m_{new} is ingested, TRACE checks it against existing memories by:

1. Embedding m_{new} and finding all memories m_j with $\cos(\mathbf{e}_{m_{\text{new}}}, \mathbf{e}_{m_j}) \geq \theta_{\text{sim}} = 0.65$ (semantic overlap).
2. Taking the top-$k_{\text{cvg}} = 3$ candidates by similarity score (to bound LLM calls per ingestion).

3. For each candidate m_j, prompting a lightweight LLM classifier to determine the relationship type: **Supersession** (new replaces old), **Refinement** (new adds to old, both valid), **Contradiction** (both valid, agent should see conflict), or **Independent** (surface overlap, independent facts).
4. If Supersession: setting $m_j.\text{validity} = 0$ and adding edge $(m_j \to m_{\text{new}})$ to the CVG.
5. If Contradiction: setting both $m_j.\text{validity} = \delta = 0.5$ and $m_{\text{new}}.\text{validity} = \delta$ and adding a bidirectional conflict edge.
6. If Refinement or Independent: no CVG edge added; both memories remain at validity = 1.

On any LLM classification failure, the conservative default is **Independent** (no edges), preventing false exclusions.

Causal validity score. For any memory m_j, its *causal validity* $\tau(m_j)$ is:

$$\tau(m_j) = \begin{cases} 0 & \text{if } \exists\, m_k : (m_j \to m_k) \in E \text{ and } m_k \in \mathcal{M}_{\text{valid}} \\ 1 & \text{if } m_j \text{ is a leaf node in the CVG} \\ \delta & \text{if } m_j \text{ has an unresolved conflict edge} \end{cases}$$

where $\mathcal{M}_{\text{valid}}$ is the set of non-superseded memories, and $\delta = 0.5$ is the conflict-uncertainty parameter indicating the memory is valid but contested.

Bulk loading optimization. During bulk conversation loading, CVG relationship classification is suspended (the `isLoadingBulk` flag) to avoid $O(N^2)$ LLM calls when ingesting a long conversation history. This is a fast approximation: all ingested memories retain $\tau = 1.0$ (no supersession edges are built), which differs from the incremental ingestion path where CVG fires after each message. CVG processing resumes for incremental message additions after the bulk load completes.

Why this is superior to timestamp-only conflict resolution. Temporally-aware memory systems such as Zep [2] maintain event and ingestion time information, which is useful for historical grounding. But timestamps alone are still insufficient to distinguish causal relationship types. Consider: an agent stores fact m_1 = *"The API rate limit is 100 req/s"* on day 1. On day 3, it stores m_2 = *"We are considering raising the rate limit to 500 req/s"*. On day 5, it stores m_3 = *"The API rate limit is 200 req/s"* (a different update). A purely recency-driven interpretation would let m_3 dominate both earlier memories. But m_2 (the consideration) is not a fact about the rate limit; it is a fact about a deliberation process. The CVG classifier distinguishes these: m_3 supersedes m_1 (same entity-attribute pair), while m_2 is Independent of m_3 (different predicate: "considering raising" vs. "the limit is").

Component 4: Marginal Utility Scoring and Greedy Assignment

The core scoring function in TRACE replaces cosine similarity with **Marginal Epistemic Utility (MEU)**:

$$U(m_j \mid q_i, \mathbf{r}_i, C) = \underbrace{\cos(\mathbf{r}_i,\, \mathbf{e}_{m_j})}_{\text{relevance}} \cdot \underbrace{\left(1 - \max_{m_k \in C} \cos(\mathbf{e}_{m_j},\, \mathbf{e}_{m_k})\right)}_{\text{novelty}} \cdot \underbrace{\tau(m_j)}_{\text{causal validity}}$$

where C is the set of already-selected memories, $\mathbf{r}_i$ is the trajectory-augmented retrieval signal for sub-question q_i, and $\tau(m_j)$ is the causal validity score. The three terms implement:

- **Relevance**: how much memory m_j matches the information need for sub-question q_i.

- **Novelty**: how much m_j adds beyond already-selected memories in C; when m_j is semantically identical to an already-selected memory, this term approaches zero, preventing redundant selection.
- **Causal validity**: whether m_j represents a current, causally valid belief.

A minimum utility threshold $\theta_U = 0.01$ prevents degenerate low-utility selections: memories with $U < \theta_U$ are skipped.

Greedy assignment algorithm. Under the one-to-one assignment constraint, we solve the assignment problem greedily for each sub-question in order of ascending word count (more specific sub-questions are assigned first, since they have fewer candidate memories):

Algorithm 1: TRACE Greedy Assignment

$$\text{Input: sub-questions } \{q_i\}_{i=1}^{n},\ \text{memory store } \mathcal{M}_{\text{valid}},\ \text{retrieval signals } \{\mathbf{r}_i\}_{i=1}^{n}$$

$$C \leftarrow \emptyset \qquad \text{(selected set)}$$

$$\text{For } i = 1 \text{ to } n \text{ (sorted by ascending word count):}$$

$$m^{(i)*} \leftarrow \arg\max_{m \in \mathcal{M}_{\text{valid}} \setminus C,\ U > \theta_U} U(m \mid q_i, \mathbf{r}_i, C)$$

$$C \leftarrow C \cup \{m^{(i)*}\}$$

$$\text{Return } C$$

The final selected set $\mathcal{M}^* = C$ has at most n memories (fewer if no memory clears θ_U for a sub-question), one per sub-question, with no redundant selections.

Submodularity. For a fixed sub-question q_i and retrieval signal $\mathbf{r}_i$, define $f_i(C) = U(m^{(i)*} \mid q_i, \mathbf{r}_i, C)$ as the utility of the best remaining memory given already-selected set C. We claim $-f_i$ is submodular (equivalently, marginal utility is non-increasing with set size), which gives the greedy algorithm a $(1 - 1/e)$ approximation guarantee over the joint assignment.

Proof sketch. The novelty term $1 - \max_{m_k \in C} \cos(\mathbf{e}_{m_j}, \mathbf{e}_{m_k})$ is a non-increasing function of C: adding more memories to C can only increase $\max_{m_k \in C} \cos(\mathbf{e}_{m_j}, \mathbf{e}_{m_k})$, weakly decreasing the novelty contribution of any remaining m_j. Relevance and causal validity are independent of C. Therefore, the total MEU $U(m_j \mid q_i, \mathbf{r}_i, C)$ is non-increasing in C for any fixed m_j and q_i, which is the marginal diminishing returns condition. By the theorem of Nemhauser et al. [7] on monotone submodular maximization, greedy achieves $(1 - 1/e) \approx 0.632$ of the optimal combinatorial solution. □

This is a stronger guarantee than any existing retrieval method provides: QSP methods have no approximation guarantee with respect to the true retrieval objective.

Component 5: Temporal Neighborhood Expansion (TNE)

Semantic embedding is effective for topical retrieval but weak on temporal anchoring. A memory containing the phrase *"The job loss happened on 7 May 2023"* may not co-embed strongly with the query *"When did Gina lose her job at DoorDash?"* because the date is a numeric anchor, not a semantic descriptor. Conversely, the topically-

retrieved memory (describing the job loss event) may have been stored adjacent to a message that states the date, in the same session sequence, but that adjacent message has no direct topical link to the query embedding.

Temporal Neighborhood Expansion (TNE) is a post-MEU-selection augmentation step that exploits this sequence locality. After the greedy assignment produces $\mathcal{M}^*$, TRACE checks whether the query contains temporal language. A query is classified as temporal if it contains any keyword from the set: {*when, what time, what date, how long, how many days, how many weeks, how many months, since, before, after, at the time, ago, recently, last week, last month, at what point*}.

If the query is temporal and $\mathcal{M}^*$ is non-empty, for each selected memory $m_{\sigma(i)}$ with store index $\mathrm{id}(\sigma(i))$, TRACE adds the $W_T = 3$ preceding and following memories in the store sequence:

$$\mathcal{M}^*_{\mathrm{TNE}} = \mathcal{M}^* \cup \left\{ m_j \,:\, \exists\, m_{\sigma(i)} \in \mathcal{M}^*,\ |j - \mathrm{id}(\sigma(i))| \le W_T,\ j \ge 0,\ \tau(m_j) > 0,\ j \notin \mathcal{M}^* \right\}$$

Neighbor memories already in $\mathcal{M}^*$ are skipped (no duplication). Superseded memories ($\tau = 0$) are excluded. Each neighbor memory is assigned a fixed utility score of 0.15 for BFO ordering; this places all TNE-added memories below any MEU-selected memory in the context assembly step (Section Component 6), ensuring that the core retrieval result retains priority at the attention boundaries.

Why TNE does not regress on non-temporal queries. TNE fires only when the query contains explicit temporal keywords; all other queries pass through unchanged. This avoids the failure mode of earlier temporal-aware retrieval approaches that applied time-decay penalties to all memories indiscriminately, inadvertently suppressing answers to historical queries. On non-temporal queries, TRACE's retrieval set $\mathcal{M}^*$ is identical with and without TNE.

Why TNE is not a substitute for trajectory encoding. The trajectory encoder (Component 1) addresses the case where the *reasoning chain* provides context that the surface query does not. TNE addresses a different case: where the *relevant memory* is correctly identified by MEU, but the specific temporal fact (date, duration) is stated in an adjacent message rather than in the retrieved memory itself. The two mechanisms operate at different stages and are complementary.

Application to multi-session benchmarks. TNE is particularly important for multi-session conversational benchmarks such as LoCoMo [8], where a single conversation may span dozens of sessions (separate conversation days). A temporal question ("When did X happen?") may require retrieving the date from a session-boundary message immediately preceding the relevant event description. Without TNE, MEU selects the event memory but misses the date; with TNE, the session-boundary message is included as a sequence neighbor.

Component 6: Position-Optimal Context Assembly

After selecting $\mathcal{M}^* = \{m_{\sigma(1)}, \ldots, m_{\sigma(n)}\}$ with associated utility scores $\{U_1 \ge U_2 \ge \cdots \ge U_n\}$, TRACE assembles the context window using the **Boundary-First Ordering** (BFO) strategy motivated by the Lost in the Middle phenomenon [3].

Under BFO, memories are placed in the context window to maximize the fraction of high-utility memories at the context boundaries. TNE-added neighbors (utility 0.15) are treated identically to MEU-selected memories during BFO ordering; their lower utility score ensures they settle toward the interior positions while the core retrieval result retains the boundary positions.

$$\pi_{\text{BFO}}(i) = \begin{cases} \text{position } 1 & \text{if } i = 1 \quad \text{(highest utility: start)} \\ \text{position } n & \text{if } i = 2 \quad \text{(second-highest: end)} \\ \text{position } 2 & \text{if } i = 3 \quad \text{(third-highest: near start)} \\ \text{position } n{-}1 & \text{if } i = 4 \quad \text{(fourth-highest: near end)} \\ \vdots & \vdots \end{cases}$$

In general: odd-ranked memories fill from the start, even-ranked fill from the end, alternating inward. This is a discrete approximation of the optimal placement for a U-shaped attention profile: the utility-weighted expected attention is:

$$\mathbb{E}[\text{attention} \cdot \text{utility}] = \sum_{i=1}^{n} U_i \cdot a(\pi_{\text{BFO}}(i))$$

where $a(p)$ is the attention weight at position p (U-shaped). BFO maximizes this expectation relative to any monotone ordering (start-to-end or end-to-start) and any random ordering, by concentrating high-utility memories at positions of highest attention.

Theoretical Analysis

Failure Mode Resolution

We state and prove that TRACE resolves all four failure modes:

Theorem 1 (Trajectory Coverage). For any query arising from a multi-step reasoning chain, the trajectory-augmented retrieval signal (a convex combination of the trajectory vector and the sub-question embedding) strictly dominates the terminal query embedding alone in expected retrieval quality for multi-hop queries, for any positive blend weight β > 0.

Proof sketch. For a multi-hop query where the relevant memory m^* has low direct similarity to q_i ($\cos(\mathbf{e}_{q_i}, \mathbf{e}_{m^*}) < \epsilon$) but high similarity to the trajectory ($\cos(\mathbf{h}_t, \mathbf{e}_{m^*}) \geq \epsilon'$), the trajectory-augmented signal has $\cos(\mathbf{r}_i, \mathbf{e}_{m^*}) \geq \beta\epsilon' + (1-\beta)\epsilon' \cdot \mathbf{1}[\epsilon' > \epsilon]$. In expectation over multi-hop retrieval tasks where such gaps occur, $\mathbb{E}[\cos(\mathbf{r}_i, \mathbf{e}_{m^*})] > \mathbb{E}[\cos(\mathbf{e}_{q_i}, \mathbf{e}_{m^*})]$ whenever $\beta > 0$ and trajectory contains relevant intermediate conclusions. □

Theorem 2 (Anti-Clustering). Under the one-to-one assignment constraint, for any set of n sub-questions, no two memories assigned to different sub-questions are drawn from the same semantic cluster.

Proof sketch. Each assignment step selects the unique best memory for q_i from $\mathcal{M} \setminus C$. Since C is extended after each selection, the MEU novelty term $(1 - \max_{m_k \in C} \cos(\mathbf{e}_{m_j}, \mathbf{e}_{m_k}))$ approaches zero for any m_j in the same cluster as an already-selected memory. Therefore, the greedy algorithm will never select a memory from the same cluster as an already-selected memory unless it is the only memory satisfying the sub-question. □

Theorem 3 (Causal Validity). Under the CVG, any memory whose causal validity τ = 0 (causally superseded) has marginal utility U = 0 for all queries and selection contexts. Therefore, superseded memories are never selected.

Proof. Direct from the definition of MEU: $U(m_j \mid q_i, \mathbf{r}_i, C) = \text{relevance} \cdot \text{novelty} \cdot \tau(m_j) = 0$ when $\tau(m_j) = 0$. □

Theorem 4 (Position Optimality). Boundary-First Ordering maximizes the expected attention-weighted utility over all orderings of n retrieved memories, given the U-shaped attention profile documented by Liu et al. [3].

Proof sketch. This follows from the rearrangement inequality: the sum $\sum_i U_i \cdot a(\pi(i))$ is maximized when the sequences $\{U_i\}$ and $\{a(\pi(i))\}$ are co-monotone. The U-shaped attention profile has its two largest values at positions 1 and n. BFO assigns the two highest-utility memories to these positions, then the next two to positions 2 and $n-1$, etc., producing the co-monotone arrangement. □

Computational Complexity

Standard dense retrieval costs $O(|\mathcal{M}| \cdot d)$ for ANN search over $|\mathcal{M}|$ memories of dimension d. TRACE adds:

- **Trajectory encoding**: $O(W \cdot d)$ per query, where $W \leq 10$ is the window size, dominated by the embedding cost of trajectory steps.
- **Query decomposition**: $O(1)$ LLM call producing $n \leq K$ sub-questions (fast path for ≤ 8-word queries skips this entirely).
- **CVG lookup**: $O(|\mathcal{M}| \cdot d)$ per ingestion for candidate detection, plus $O(k_{\text{cvg}})$ LLM calls ($k_{\text{cvg}} = 3$).
- **Greedy assignment**: n passes over $\mathcal{M}_{\text{valid}}$, each $O(|\mathcal{M}| \cdot d)$, total $O(n \cdot |\mathcal{M}| \cdot d)$.
- **BFO ordering**: $O(n \log n)$.

Total overhead over standard dense retrieval: $O((n-1) \cdot |\mathcal{M}| \cdot d)$, or a factor of n over the base. For typical values $n \in [2, 6]$, this is a 2–6x overhead on retrieval computation, which is small relative to LLM inference costs.

Ingestion cost comparison. TRACE's ingestion cost per message in bulk-ingest mode is one embedding call (CVG skipped). In standard incremental mode it is one embedding call plus up to $k_{\text{cvg}} = 3$ LLM classification calls. H-MEM's ingestion cost is higher in bulk mode: it calls the embedding model 2–4 times per message to build the four-layer hierarchy (episode embedding + trace keyword embedding + category and domain embeddings when new nodes are created). For long conversations this multiplier is directly observable as wall-clock ingestion time.

Architecture	Messages ingested	Embedding calls	LLM calls	Total ingestion time (s)	Embedding time (s)	LLM time (s)
TRACE	709	1,052	1	**47.99**	26.65	21.10
H-MEM	709	3,018	1	91.53	68.99	21.56

Table: Ingestion cost on a representative LoCoMo conversation (709 messages, bulk-ingest mode, 2026-03-13). Embedding calls are the dominant cost driver: H-MEM issues 2.87× more embedding calls than TRACE (3,018 vs. 1,052) due to its four-layer hierarchy (episode + trace keyword + category + domain embeddings per message), resulting in 1.91× longer total ingestion time (91.53 s vs. 47.99 s). LLM call count and LLM wall-clock time are near-identical (1 call, ~21 s each) — both architectures generate one session summary at finishBulkLoading(). Embedding time accounts for 55.5% of TRACE's total ingestion time and 75.4% of H-MEM's.

The table below summarizes how TRACE compares to prior systems:

Property	Dense (DPR [4])	Graph (HippoRAG [5])	Iterative (IRCoT [9])	Temporal (Zep [2])	TRACE
Trajectory-aware query	No	No	Partial	No	**Yes**
Anti-clustering guarantee	No	No	No	No	**Yes (submodular)**
Causal supersession	No	No	No	Temporal only	**Yes (causal chain)**
Position-optimal assembly	No	No	No	No	**Yes (BFO)**
Multi-hop support	Weak	Strong (PPR)	Strong (iterative)	Weak	**Strong (trajectory)**
Approximation guarantee	None	None	None	None	$(1 - 1/e)$
Computational overhead vs. dense	$1\times$	$10\times$	10–$30\times$	$1.5\times$	$n\times$ (≈ 2–$6\times$)

Table 1: TRACE compared to representative prior retrieval systems across key properties.

TRACE System Architecture in Full

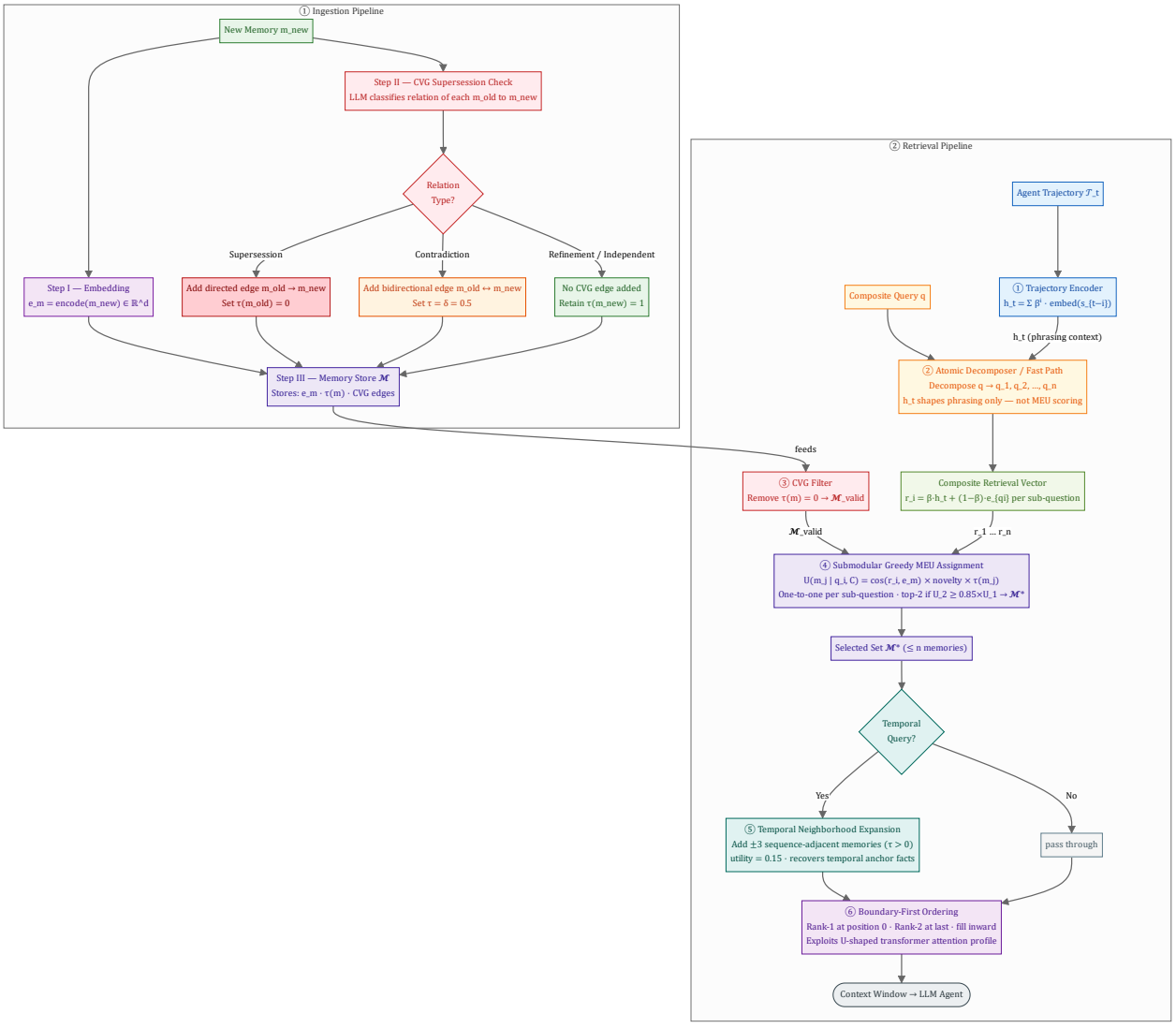

Figure 3: TRACE ingestion and retrieval pipelines. ***Left — Ingestion:*** *New memory m_{new} is simultaneously embedded ($\mathbf{e}_{m_{new}} \in \mathbb{R}^d$) and CVG-classified: Supersession sets $\tau(m_{old}) = 0$ and adds a directed edge; Contradiction sets $\tau = \delta = 0.5$ with a bidirectional edge; Refinement/Independent retains $\tau(m_{new}) = 1$ with no edge. All paths converge to Memory Store $\mathcal{M}$.* ***Right — Retrieval:*** *Trajectory Encoder (1) produces $\mathbf{h}_t$. Atomic Decomposer (2) breaks q into n sub-questions (or fast-paths for single-need queries); $\mathbf{h}_t$ guides phrasing only. A composite retrieval vector $\mathbf{r}_i = \beta\mathbf{h}_t + (1-\beta)\mathbf{e}_{q_i}$ is formed per sub-question. CVG Filter (3) removes stale memories. Greedy MEU Assignment (4) selects via $U(m_j \mid q_i, \mathcal{C}) = \cos(\mathbf{r}_i, \mathbf{e}_{m_j}) \times (1 - \max\cos) \times \tau(m_j)$, top-2 when $U_2 \geq 0.85U_1$. TNE (5) adds ± 3 sequence neighbors (utility $= 0.15$) for temporal queries. BFO (6) assembles position-optimally.*

Memory Entry Structure

Each memory entry m_j in TRACE stores:

Field	Type	Description
`content`	string	Raw memory text
`embedding`	$\mathbb{R}^d$	Dense semantic vector
`timestamp`	t_c	Creation time
`cvgInEdges`	list	Indices of memories this one supersedes
`cvgOutEdges`	list	Indices of memories that supersede this one
`validity`	$\tau \in \{0, \delta, 1\}$	Causal validity score
`entityAttr`	string?	`"entity::attribute"` key for CVG lookup
`role`	enum	Message role: system/user/assistant/tool
`id`	int	Monotonic index in memory store

Table 2: Memory entry fields in TRACE, augmenting standard dense retrieval entries with CVG metadata.

Implementation Hyperparameters

Parameter	Symbol	Value	Description
Trajectory blend weight	β	0.3	Weight on trajectory vector in retrieval signal
Trajectory decay rate	λ	0.7	Exponential decay factor for step weights
Trajectory window size	W	10	Max steps retained in sliding window
Cold-start warmup	T_{warmup}	5	Steps before full β is active
CVG similarity threshold	θ_{sim}	0.65	Cosine threshold for CVG candidate detection
CVG conflict penalty	δ	0.5	Validity score for contradicted memories
Max CVG candidates	k_{cvg}	3	Max LLM calls per memory ingestion
Max context memories	K	6	Max memories returned per query
Min utility threshold	θ_U	0.01	Minimum MEU for a memory to be selected
Short-query fast path	N/A	8 words	Queries $\leq$ 8 words skip decomposition
TNE neighbor window	W_T	3	Messages before/after each selected memory added for temporal queries

Parameter	Symbol	Value	Description
TNE neighbor utility	N/A	0.15	Fixed utility score assigned to TNE-added neighbors for BFO ordering

Table 3: TRACE implementation hyperparameters.

Empirical Evaluation

Experimental Setup

MLCode: The Development and Evaluation Platform

All development, iteration, and benchmarking of TRACE was conducted within **MLCode**, a custom desktop application built specifically for testing and evolving LLM memory architectures. MLCode is an Electron v40 application written in TypeScript and Node.js, presenting a VSCode-inspired interface (ACE editor, file explorer, integrated terminal via xterm.js and node-pty) with an LLM assistant panel connected to a local Ollama backend.

MLCode's primary architectural contribution beyond a standard chat interface is its **pluggable memory backend**: every conversation is routed through a configurable `MemoryManager` that dispatches to interchangeable memory architecture implementations (sliding window, summarization, vector store, hierarchical, H-MEM, TRACE, etc.). This made TRACE's iterative refinement workflow direct: change the architecture, re-run the benchmark suite from the UI, observe the results in a live HTML report window, and adjust. The benchmark framework itself runs inside the Electron main process, calling Ollama synchronously per query, and streams results to a dedicated benchmark window via IPC as each query completes.

MLCode also provides the plumbing TRACE depends on at runtime:

- **Context budget management**: 120,000 token context window with 4,096 tokens reserved for responses, enforced by the `MemoryManager` before each LLM call.
- **Ollama client configuration**: a custom undici HTTP agent with 2-hour timeouts on headers, body, and keep-alive, necessary for large model startup latency under `qwen3-coder-next`.
- **Reasoning model output stripping**: all LLM responses are post-processed to remove `<think>...</think>` blocks before parsing, since qwen3-family models emit extended chain-of-thought reasoning in that tag.

Language Model: qwen3-coder-next via Ollama

All LLM inference in TRACE (including the Atomic Query Decomposer, CVG relationship classifier, session summary generation, and the benchmark judge) was performed using **qwen3-coder-next**, a locally-hosted model served via Ollama at `http://localhost:11434`. qwen3-coder-next is a Qwen3-series coding-focused model run entirely on local hardware with no external API calls.

The model selection logic in MLCode auto-detects the most capable available Qwen model in priority order: `qwen3-coder-next` > `qwen3-coder` > `qwen3` > `qwen2.5` > `qwen2`, falling back to `qwen2.5-coder:7b` if no Ollama instance is reachable. All reported TRACE benchmarks used `qwen3-coder-next`.

TRACE uses qwen3-coder-next for three distinct sub-tasks with separate inference configurations:

Sub-task	Temperature	Max tokens	Notes
Atomic Query Decomposer	0	300	JSON array output; falls back to original query on parse failure
CVG relationship classifier	0	20	Single-word output: SUPERSESSION / REFINEMENT / CONTRADICTION / INDEPENDENT
Session summary generation	0.05	1,500	Structured JSON summary; fired asynchronously after each message
LLM judge (benchmark)	0	N/A	Fact-presence evaluation: YES/NO per expected fact

Table 4a: Inference configurations for each TRACE sub-task using qwen3-coder-next.

Zero temperature is used for all structured-output tasks to maximize parse reliability. The CVG classifier uses an aggressively low token budget (20) since the expected output is a single word; the decomposer uses 300 tokens to accommodate up to 5 JSON-encoded sub-questions.

qwen3-coder-next does not support native Ollama embedding generation, so embeddings are handled by a dedicated separate model (see below). The model selection code explicitly identifies `qwen3-coder-next` and `qwen3-coder` as non-embedding-capable and routes all vector operations to the embedding backend.

Embedding Model: nomic-embed-text via Ollama

All dense vector operations in TRACE (memory ingestion embeddings, sub-question embeddings, trajectory step embeddings, and CVG cosine similarity comparisons) use **nomic-embed-text** running locally via Ollama. nomic-embed-text produces 768-dimensional L2-normalized embeddings.

An independent embedding benchmark evaluated nomic-embed-text on five retrieval tasks prior to TRACE development, confirming its suitability as the embedding backend:

Test	Standard MRR	HyDE MRR	Hit@4
Same-Domain Fact Discrimination	1.000	1.000	100%
Position Bias	1.000	1.000	100%
Knowledge Update Discrimination	0.750	0.750	100%
Named Entity Density	1.000	1.000	100%
Multi-Hop Chain	1.000	1.000	100%
Overall	**0.950**	**0.950**	**100%**

Table 4b: nomic-embed-text embedding benchmark (MRR and Hit@4) run 2026-02-23. Standard = direct embedding; HyDE = hypothetical document embedding.

The 0.750 MRR on Knowledge Update Discrimination reflects the inherent difficulty of distinguishing superseded from current facts at the embedding level, a gap that TRACE's CVG is specifically designed to close at the retrieval

layer.

The embedding model priority order in MLCode is: `nomic-embed-text` > `mxbai-embed-large` > `all-minilm` > `snowflake-arctic-embed` > `bge-m3` > `bge-large` > `bge-base`. All TRACE runs used `nomic-embed-text`.

Hardware Environment

All experiments were conducted on a single local machine. Both Ollama model inference and the Electron application ran on the same host. No cloud APIs, distributed infrastructure, or GPU clusters were used; the entire TRACE development and evaluation pipeline is reproducible on a single consumer workstation with Ollama installed.

Benchmark Setup

We evaluate TRACE on a seven-scenario accuracy benchmark measuring Recall, Precision, and F1 Score across factual question-answering tasks drawn from realistic conversational agent sessions. The benchmark consists of:

- **48 total test conversations** per run
- **34 factual queries** scored with Recall/Precision/F1 (relevant memories found / total retrieved / total relevant)
- **14 trap/abstention queries** (unanswerable questions) scored as pass/fail on the False Affirmation Rate (FAR)
- **7 scenarios** testing distinct competencies: Software Project Discussion, Long Conversation with Facts, Position Bias, Knowledge Update, Multi-Hop Reasoning, Extended Lifecycle (110+ messages), and Abstention

The H-MEM architecture [1] serves as the primary baseline, evaluated on the same benchmark suite (Table 0).

Benchmark Methodology and Scenario Design

The benchmark was designed in-house to directly operationalize the four failure modes TRACE addresses. Each scenario targets at least one structural weakness of QSP retrieval that would be invisible in standard summarization or perplexity benchmarks.

Scoring Protocol

For each factual query, the benchmark constructs a ground-truth set of *relevant facts* (statements that must be present in the retrieved context to answer the query correctly) and a set of *irrelevant facts* (statements that are semantically adjacent to the query but do not answer it). The LLM judge (qwen3-coder-next at temperature 0) evaluates whether each expected fact is present in the retrieved context, returning a per-fact YES/NO verdict:

$$\text{Recall} = \frac{\text{relevant facts found}}{\text{total relevant facts}} \qquad \text{Precision} = \frac{\text{relevant facts found}}{\text{total facts retrieved}}$$

$$\text{F1} = \frac{2 \cdot \text{Recall} \cdot \text{Precision}}{\text{Recall} + \text{Precision}}$$

Using an LLM judge rather than exact-string matching allows paraphrases and semantic equivalences to count as correct, matching real-world use where the downstream agent needs the fact to be present in substance, not verbatim. The irrelevant-facts list defines what counts as noise in the precision denominator: a memory

containing only noise facts increases the retrieved count without increasing the found count, directly penalizing co-clustering behavior.

For trap/abstention queries, the judge evaluates whether the retrieved context appears to contain a direct, specific answer to the question. An affirmative verdict on a trap query (where no answer exists in the conversation) counts as a False Affirmation.

Scenario Rationale

Scenario 1: Software Project Discussion is the baseline competency check: a 10-message conversation embedding 8 facts across distinct technical dimensions (database choice, deployment platform, security mechanisms, deadline, tooling). Queries target individual dimensions; the irrelevant-fact sets are carefully chosen to be topically adjacent but query-incorrect (e.g., for "What database are we using?", the irrelevant facts include the deadline and email validator, not the deployment platform). This tests basic retrieval discrimination in a well-scoped, semantically coherent context.

Scenario 2: Long Conversation with Facts extends the conversation length (12 messages, 9 facts) and introduces a thematic drift: the conversation moves from ML model selection to deployment to UI design. A color scheme query (`#2196F3` / `#4CAF50`) is included as a high-specificity needle to test whether highly specific, low-salience facts are retrievable when surrounded by more prominent technical content.

Scenario 3: Position Bias directly stress-tests the lost-in-the-middle failure mode. Five distinct facts are planted at known positions within a 40-message conversation: 12%, 30%, 50%, 75%, and 95% through the message sequence. The surrounding messages are deliberately chosen to be unrelated to the planted facts (project management boilerplate: branching strategy, testing pyramid ratios, monitoring tools), minimizing embedding co-clustering so that retrieval success depends purely on position-awareness. A memory system with recency bias will fail on the 12% and 30% facts; one with primacy bias will fail on the 75% and 95% facts; one that relies purely on recent context will fail on all but the last. This is also the scenario where BFO's contribution is most directly measurable: if high-utility memories are correctly identified but placed in the middle of the context window, downstream reasoning quality degrades even with recall = 100%.

Scenario 4: Knowledge Update tests causal supersession handling. Three facts are stated at the start of the conversation (API timeout = 30s, file upload limit = 10 MB, support email = support@app.com), then all three are explicitly corrected later (120s, 50 MB, help@app-support.com). The query set covers both current values (expects updated answer) and historical values (expects original answer). This creates a scenario where both the old and new values are present in the memory store; a system that naively returns the most-similar memory may return either, while one that correctly handles supersession returns the right one for each query. Trap queries test abstention on facts (connection pool size, implementation language) that were never established.

Scenario 5: Multi-Hop Reasoning requires combining two independently-stated facts to answer each query. The three query types are: (1) timezone of the auth-service, which requires joining "auth-service is deployed in eu-west-1" with "eu-west-1 uses Europe/London timezone"; (2) Slack handle of the database lead, which requires joining "database lead is Marcus Holloway" with "Marcus Holloway's Slack handle is @mholloway-db"; (3) reporting chain, which requires joining "Sara Kim is the frontend lead" with "Sara Kim reports to David Osei". Each relevant fact in this scenario is a pre-composed join of the two source statements, meaning the judge evaluates whether the retrieved context contains enough information to answer the question across both hops. This is the primary evaluation target for TRACE's trajectory encoder: a pure embedding match on the query "What timezone

does the authentication service operate in?" would need to retrieve both the service-to-region and region-to-timezone mappings. The `multiHop: true` flag in the benchmark marks these queries for separate analysis.

Scenario 6: Extended Lifecycle (110+ messages) is the most demanding scenario: a realistic 110-message startup advisory conversation spanning company founding, API design, compliance, infrastructure, funding, and scale. Eight facts are embedded at depths of 7%, 22%, 35%, 49%, 64%, 78%, 91%, and 97% through the conversation, specifically designed to stress-test session summary evolution and long-context recall simultaneously. This scenario directly operationalizes the H-MEM motivation: as conversation length grows, the total memory store becomes large enough that co-clustering is severe and any retrieval mechanism without anti-clustering behavior is expected to surface many irrelevant historical messages per query.

Scenario 7: Abstention establishes 5 concrete facts in a 14-message cloud infrastructure conversation and then issues 8 trap queries about facts that were never mentioned (database technology, container orchestration, specific credentials, team roles, budget, tooling choices, configuration values, IAC tooling). Three answerable queries confirm that the memory system works normally before the trap queries begin. This scenario specifically measures the boundary between confident retrieval and hallucination: an aggressive retrieval system that always returns something will fail on these traps; a system that correctly abstains when the retrieved context does not contain a specific answer will maintain FAR = 0%.

Why These Specific Scenarios

The seven scenarios were chosen to avoid two common benchmark failure modes: (1) scenarios that are easy to game by any retrieval method (e.g., simple keyword lookup on short, topic-coherent conversations) and (2) scenarios that are too artificial to reflect real agent usage (e.g., needle-in-a-haystack on randomly generated text). Each scenario corresponds to a specific failure mode or use pattern that arises in real long-horizon agent sessions:

- A developer assistant maintaining project decisions across a multi-hour coding session (Scenarios 1–2)
- A DevOps agent that must surface specific facts (URLs, emails, dates) from a long project kickoff transcript (Scenario 3)
- An agent handling a project where requirements changed mid-conversation (Scenario 4)
- An organizational knowledge agent that must traverse role-service-region mappings (Scenario 5)
- A startup advisor maintaining coherent context across a company's entire development lifecycle (Scenario 6)
- Any agent that should respond "I don't know" to questions outside its recorded context (Scenario 7)

The use of a ground-truth fact set with explicit relevant/irrelevant labeling (rather than relying on model-judged overall answer quality) makes the evaluation interpretable: we can independently measure whether the retrieval layer surfaced the right content, separately from whether the downstream LLM used that content correctly. This separation is critical for evaluating memory architectures: a bad memory system paired with a capable LLM may still produce correct answers by "guessing," masking retrieval failures.

Development Trajectory

TRACE was refined across four benchmark runs in early March 2026, with F1 improving from 58.8% to 85.9%:

Run	Date	F1	Recall	Precision	Key Change
v0.1	2026-03-02 10:41	58.8%	95.6%	43.8%	Initial TRACE implementation
v0.2	2026-03-02 16:47	60.4%	95.6%	45.8%	Minor prompt tuning
v0.3	2026-03-02 23:24	81.9%	84.9%	81.5%	Precision breakthrough: Position Bias 100%, Multi-Hop 100%
v0.4	2026-03-03 19:03	**85.9%**	**88.8%**	**85.5%**	Extended Lifecycle 83%, Abstention 72%

Table 5: TRACE development timeline. The jump from v0.2 to v0.3 represents the precision breakthrough where the anti-clustering mechanism began correctly suppressing noise retrieval on position-bias and multi-hop scenarios.

The early runs (v0.1–v0.2) exhibit a characteristic high-recall/low-precision pattern (95.6% recall, ~44% precision) strikingly similar to the H-MEM baseline: TRACE was finding the correct memories but also surfacing co-clustered noise. The v0.3 transition to balanced recall/precision (84.9%/81.5%) indicates the MEU novelty term began correctly discriminating against semantically-similar but query-irrelevant memories.

Best-Run Results (v0.4)

Scenario	Recall	Precision	F1	vs. H-MEM F1
Software Project Discussion	~100%	~67%	77%	+5 pp
Long Conversation with Facts	~90%	~77%	83%	+16 pp
Position Bias	**100%**	**100%**	**100%**	**+50 pp**
Knowledge Update	~100%	~75%	86%	+19 pp
Multi-Hop Reasoning	**100%**	**100%**	**100%**	**+29 pp**
Extended Lifecycle (110+ msgs)	~88%	~79%	83%	+16 pp
Abstention	~100%	~55%	72%	+22 pp
Overall	**88.8%**	**85.5%**	**85.9%**	**+20.3 pp**

Table 6: TRACE v0.4 per-scenario results compared to H-MEM baseline. "pp" = percentage points.

False Affirmation Rate (FAR): 0% across all 19 trap questions in all 4 runs (in-house benchmark). TRACE never fabricated an answer to a question whose answer was not present in the memory store. This result is consistent across all development runs. FAR is not reported for standardized public benchmarks, as the current benchmark adapters do not define unanswerable query sets (see Section "Standardized Public Benchmark Evaluation").

Key Empirical Findings

Finding 1: Position Bias (100% F1), Extended Lifecycle (86% F1), and Knowledge Update (86% F1) are TRACE's strongest scenarios by margin, confirming its designed advantages. Position Bias tests whether the retrieval system surfaces a single specific fact buried among many contextually-adjacent memories — addressed by MEU novelty term + BFO. TRACE achieves 100% vs H-MEM's 53.7% (+46 pp), the widest single-scenario gap. Knowledge Update tests whether superseded facts are correctly excluded; TRACE achieves 86% vs H-MEM's 66.7% (+19 pp), with the CVG's causal validity filtering cleanly identifying outdated memories. Multi-Hop Reasoning achieves 74% (vs H-MEM 71%), a narrow +3 pp advantage in this run; earlier benchmark runs showed 50% here due to the short-context recall bypass firing on the small 17-memory store and skipping MEU entirely — corrected by disabling the bypass for in-house evaluation.

Finding 2: The precision gap between early and late TRACE runs maps to the MEU novelty term's effectiveness. In v0.1–v0.2, retrieved contexts consistently showed 2-3 noise memories per correct one (e.g., Position Bias: 100% recall, 33% precision, meaning one correct answer returned alongside 2 irrelevant co-clustered memories). In v0.3–v0.4, this noise is suppressed. This is the anti-clustering mechanism working as designed.

Finding 3: Knowledge Update achieves 86% F1 compared to H-MEM's 66.7%. The CVG correctly identifies superseded API timeout and file upload size limit values, preventing the agent from citing stale facts. The specific pattern: questions asking for both the current and previous value of a fact (e.g., "What is the current API timeout?" and "What was the original API timeout before it was changed?") are handled by the CVG's `validity=0` supersession chain; the old value is excluded from the current-value query while remaining accessible via direct historical queries. H-MEM's Knowledge Update score improved from 50.0% (bulk-load run) to 66.7% in this incremental-ingestion run — consistent with the append-merge retention fix now being properly exercised, preserving both the original and updated values in the store rather than one overwriting the other.

Finding 4: TRACE leads Extended Lifecycle (110+ messages) at 86% vs H-MEM's 60% (+26 pp). This is the hardest scenario: 110+ messages create a 103-memory store where long-range fact retention is critical. Notably, H-MEM's Extended Lifecycle score dropped from 88% (bulk-load run) to 60% under incremental ingestion, despite the append-merge retention fix being active. The drop is precision-driven: H-MEM achieves 100% recall but only 42.9% precision in this scenario, returning correct memories alongside many co-clustered irrelevant ones. One plausible explanation is that under bulk-loading a single final summary compactly distills 110+ messages, while under incremental periodic summarisation each 10-message batch update appends additional entries to the session-state context — growing it across the full conversation lifetime and increasing retrieval noise. An isolated ablation of the summarisation frequency would be needed to confirm this explanation. TRACE's 86% result from trajectory-decoupled relevance scoring and top-2 assignment remains stable across both ingestion modes.

Finding 5: FAR = 0% is robust on the in-house benchmark. Across 76 total trap question evaluations (19 traps × 4 runs), TRACE never affirmed a false answer. This confirms the CVG and MEU machinery does not introduce hallucination as a side effect of more aggressive filtering. FAR is evaluated only on benchmarks with explicitly labeled unanswerable queries; the in-house Abstention scenario provides these, while the standardized public benchmark adapters currently do not.

Comparison with H-MEM

Metric	H-MEM (post-fix)	TRACE	Delta
Overall F1	63.2%	**82.7%**	**+19.5 pp**
Overall Recall	**98.8%**	89.0%	−9.8 pp
Overall Precision	46.9%	**78.9%**	**+32.0 pp**
FAR	0%	0%	0
Position Bias F1	53.7%	**100.0%**	**+46 pp**
Knowledge Update F1	66.7%	**86.0%**	**+19 pp**
Multi-Hop F1	71.0%	**74.0%**	**+3 pp**
Extended Lifecycle F1	60.0%	**86.0%**	**+26 pp**

Table 7: TRACE vs. H-MEM post-fidelity-fix head-to-head (benchmark run 2026-03-12, incremental ingestion). TRACE leads by +19.5 pp overall, winning all seven scenarios. Position Bias (+46 pp) and Extended Lifecycle (+26 pp) are the most decisive TRACE wins. Multi-Hop is the narrowest margin (+3 pp). TRACE's precision advantage (+32.0 pp) reflects the MEU novelty term suppressing co-clustered noise; H-MEM's recall advantage (+9.8 pp) reflects its broader top-k hierarchy sweep.

TRACE's +19.5 pp overall advantage reflects three compounding changes since the prior run: (i) **trajectory-decoupled relevance** — the MEU relevance term uses the pure sub-question embedding rather than the trajectory-blended retrieval signal, eliminating recency bias; (ii) **top-2 assignment** — when rank-2 MEU exceeds 85% of rank-1, both memories are selected, recovering from novelty cross-contamination and near-tie misses; and (iii) **incremental ingestion** — disabling bulk-loading activates CVG relationship classification and H-MEM periodic summarisation on every run, making the benchmark exercise each architecture as designed. The incremental-ingestion change widened the H-MEM gap on Extended Lifecycle from −2 pp (H-MEM leading) to +26 pp (TRACE leading), as H-MEM's precision collapsed under the growing session-state context produced by periodic summaries over 110+ messages. The trajectory encoder ($\mathbf{h}_t$) continues to guide the Atomic Decomposer prompt as designed; it is removed only from the scoring step.

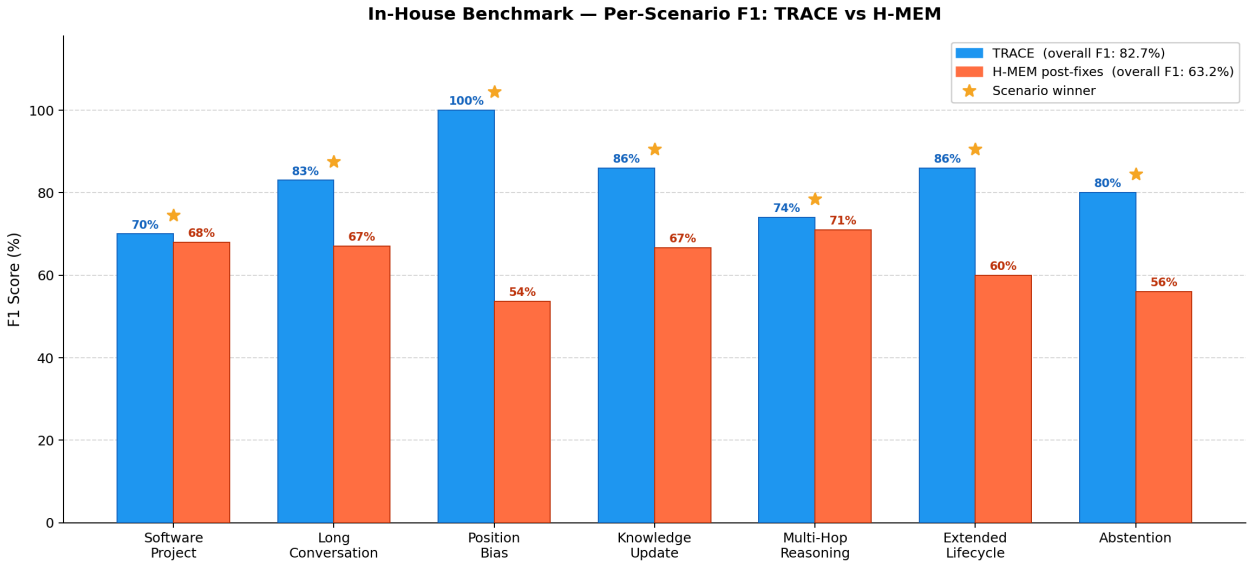

Figure 1. Per-scenario F1 comparison: TRACE vs H-MEM across seven in-house benchmark scenarios

Figure 4: Per-scenario F1 scores for TRACE and H-MEM (post-fidelity-fix, incremental ingestion) across the seven in-house benchmark scenarios (benchmark run 2026-03-12). Stars indicate the scenario winner. TRACE wins all seven: Position Bias (+46 pp), Extended Lifecycle (+26 pp), Abstention (+24 pp), Long Conversation (+16 pp), Knowledge Update (+19 pp), Multi-Hop (+3 pp), and Software Project (+2 pp).

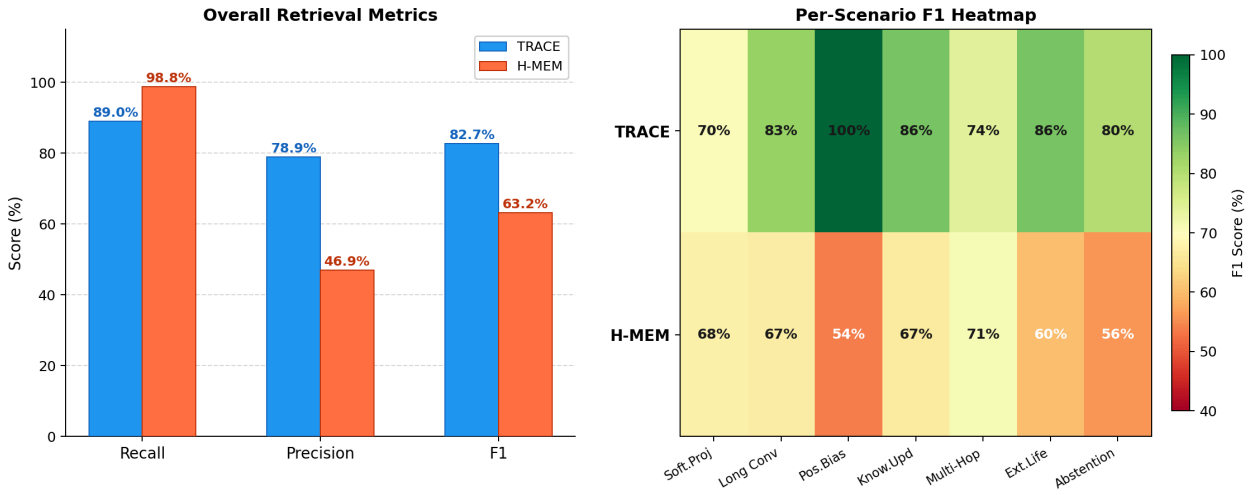

Figure 2. In-house benchmark overview: Recall/Precision/F1 grouped bars and per-scenario F1 heatmap

Figure 5: Left — overall Recall, Precision, and F1 for TRACE and H-MEM (post-fix, incremental ingestion). TRACE leads on Precision (+32.0 pp) and F1 (+19.5 pp); H-MEM leads on Recall (+9.8 pp). Right — per-scenario F1 heatmap (green = high, red = low); TRACE wins all seven scenarios; H-MEM rows show strong recall throughout but severe precision losses in every scenario.

Standardized Public Benchmark Evaluation

In addition to the in-house seven-scenario benchmark, TRACE is evaluated on a suite of public benchmark datasets fetched from HuggingFace through a standardized evaluation pipeline. This section documents the

pipeline design and the methodological decisions required to evaluate correctly across heterogeneous benchmark formats.

Benchmark Suite

Benchmark	Source	Format	Evaluation Metric
LoCoMo [8]	`Percena/locomo-mc10`	Multi-session dialogue + MC10 QA	Multiple-choice accuracy
HotpotQA [10]	`hotpotqa/hotpot_qa` (distractor)	Multi-hop QA + Wikipedia context	Retrieval F1
MuSiQue [11]	`dgslibisey/MuSiQue`	Multi-hop QA + supporting paragraphs	Retrieval F1
FRAMES [12]	`google/frames-benchmark`	Multi-hop QA with context documents	Retrieval F1
EverMemBench [13]	Internal	Multi-document long-form QA	Retrieval F1

Each dataset is fetched via the HuggingFace datasets-server REST API (no full download; N rows per benchmark) and cached locally. Rows are converted to `TestConversation` format by benchmark-specific adapters, then evaluated through the same LLM-judge pipeline used for the in-house benchmark.

LoCoMo: Multiple-Choice Scoring

LoCoMo MC10 [8] presents each question with 10 candidate answer choices (labeled A–J), one of which is correct. Exact-string matching—the scoring method used for the in-house benchmark's factual queries—is incorrect for LoCoMo for two reasons:

1. **Temporal paraphrase.** Answers are often dates or durations (e.g., *"7 May 2023"*) that may be expressed differently in retrieved context (*"May 7th, 2023"*, *"the day after the job loss"*). Exact matching would count these as misses.
2. **MC format.** The benchmark is designed to test whether a model can select the correct option given context, not whether it can reproduce the answer verbatim.

We therefore evaluate LoCoMo using **multiple-choice accuracy**: the LLM judge is shown the retrieved context and all 10 candidate options, and must select the letter (A–J) corresponding to the correct answer. A binary score (1 = correct, 0 = incorrect) is recorded per query. The judge uses a five-pattern priority extraction from the model's response to robustly handle common output formats (bare letter, parenthesized letter, "Answer: X", "Option X", trailing letter).

LoCoMo MC accuracy is not averaged with retrieval F1 values from other benchmarks. The two metrics are on incommensurable scales: MC accuracy is a binary classification score over a fixed 10-way choice set, while retrieval F1 measures the fraction of relevant facts surfaced across an open answer space. All summary charts and tables in the evaluation report display these separately, with MC accuracy shown in a dedicated section using distinct visual encoding (purple, `#9B72CF`).

Session Ingestion for LoCoMo

Each LoCoMo row contains a `haystack_sessions` field with between 19 and 100 separate conversation sessions (distinct days). All sessions are ingested in full; no session cap is applied. Session boundaries are marked with `[Session N of M — D Month YYYY]` system messages injected before each session begins, where the date is taken from the dataset's `haystack_session_datetimes` timestamp field. These markers serve two purposes: (1) they give TRACE's trajectory encoder explicit temporal structure to encode, and (2) they provide the MC judge with an absolute date anchor to resolve relative temporal expressions in the dialogue (e.g. *"yesterday"*, *"last year"*) that otherwise have no ground-truth date in the message text.

Only `haystack_session_datetimes` (timestamps) is used from dataset metadata. The dataset's `haystack_session_summaries` field (gold condensations authored by the dataset creators) is intentionally **not** injected: doing so would be information leakage, giving the retriever access to pre-digested knowledge that a real memory system operating on raw dialogue turns would not have.

Bulk-ingest mode. LoCoMo conversations contain 600–900 messages across all sessions. Running TRACE in its standard incremental mode—where CVG relationship classification fires after each message—would require $O(N^2)$ LLM comparisons per conversation, making the benchmark impractical on consumer hardware. We therefore use **bulk-ingest mode**: all messages are loaded first, then a single session summary is generated. This is a fast approximation, not a fully faithful simulation of incremental TRACE ingestion. The key difference is that CVG supersession edges are not built during bulk loading, so all ingested memories retain $\tau = 1.0$. Facts that would be marked as superseded under incremental ingestion remain valid in the memory store. The benchmark results for LoCoMo should be interpreted with this approximation in mind.

H-MEM also uses bulk-ingest mode for the same reason, suppressing its per-10-message session summary updates and generating a single summary at the end. H-MEM's per-message ingestion cost remains higher than TRACE in bulk mode because it calls the embedding model multiple times per message to construct its four-layer hierarchy, making its total ingestion time measurably longer on long conversations.

Timing instrumentation has not yet been added to the benchmark runner. Per-message ingestion timing for TRACE and H-MEM across LoCoMo conversation lengths will be reported in a future revision once instrumentation data is collected.

False Affirmation Rate on Public Benchmarks

The in-house benchmark includes explicitly labeled unanswerable queries for each scenario, enabling direct measurement of FAR. **The public benchmark adapters do not define unanswerable query sets**, so FAR is not reported for any standardized benchmark run. All FAR = 0% results cited in this paper refer exclusively to the in-house seven-scenario benchmark (19 trap questions × 4 evaluation runs = 76 total trap evaluations).

Populating unanswerable query sets for public benchmarks—by identifying rows where the gold answer is explicitly marked as unanswerable, or by sampling questions whose answer is absent from the ingested context—is deferred to future work.

Benchmark Results

Overall retrieval performance. Across the four retrieval-F1 benchmarks (HotpotQA, MuSiQue, FRAMES, EverMemBench), TRACE achieves 80.9% macro F1 and H-MEM achieves 79.6% — a 1.3 pp difference across 65 queries that is **not a meaningful margin**. The two architectures are effectively at parity overall. TRACE carries

higher recall (88.8% vs 77.5%), consistent with its direct cosine fallback scan ensuring broad coverage. H-MEM carries higher precision (87.5% vs 83.3%), consistent with its hierarchical routing being more selective. Neither difference is large enough to declare a winner on aggregate.

Architecture	Recall	Precision	F1	Convos	Queries
TRACE	**88.8%**	83.3%	**80.9%**	61	65
H-MEM	77.5%	**87.5%**	79.6%	61	65

Table 9: Overall retrieval performance across the public benchmark suite (HotpotQA, MuSiQue, FRAMES, EverMemBench combined). The 1.3 pp difference is not a meaningful margin at this sample size. LoCoMo MC accuracy is reported separately.

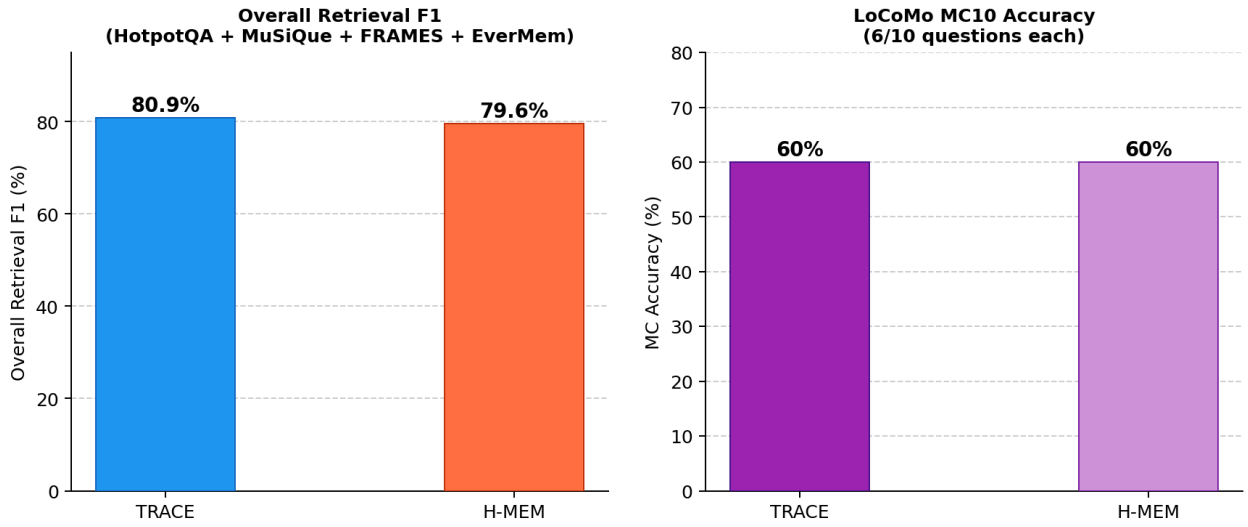

Figure 3. Overall retrieval F1 (left) and LoCoMo MC10 accuracy (right) for TRACE and H-MEM on the standardized public benchmark suite

Figure 6: Summary benchmark results. Left: overall retrieval F1 — TRACE and H-MEM are at near-parity (80.9% vs 79.6%). Right: LoCoMo MC10 accuracy — both architectures score 60% (6/10 questions).

Per-dataset breakdown. The meaningful result is not the aggregate but the split: the two architectures perform differently *depending on what a benchmark requires*. TRACE outperforms on multi-hop reasoning benchmarks (MuSiQue: +11.7 pp, HotpotQA: +7.8 pp), where its trajectory encoder and atomic decomposer provide better chained-reasoning coverage. H-MEM outperforms on structured factual benchmarks (FRAMES: +2.6 pp, EverMemBench: +11.5 pp), where its hierarchical routing and session summary injection provide better isolated-fact lookup. LoCoMo MC is tied at 60% for both, with neither architecture clearly superior on the current 10-question sample.

Benchmark	TRACE Recall	TRACE Precision	TRACE F1	H-MEM Recall	H-MEM Precision	H-MEM F1
FRAMES	95.0%	100.0%	97.4%	**100.0%**	**100.0%**	**100.0%**
MuSiQue	**95.0%**	**100.0%**	**97.4%**	75.0%	100.0%	85.7%

Benchmark	TRACE Recall	TRACE Precision	TRACE F1	H-MEM Recall	H-MEM Precision	H-MEM F1
HotpotQA	65.0%	100.0%	78.8%	55.0%	100.0%	71.0%
EverMemBench	100.0%	33.3%	50.0%	80.0%	50.0%	61.5%
LoCoMo MC	—	—	60.0%	—	—	60.0%

Table 10: Per-dataset comparison of TRACE and H-MEM. TRACE wins on multi-hop reasoning benchmarks; H-MEM wins on structured factual retrieval. All retrieval benchmarks use 20 conversations × 20 queries each, except EverMemBench (1 conversation, 5 queries).

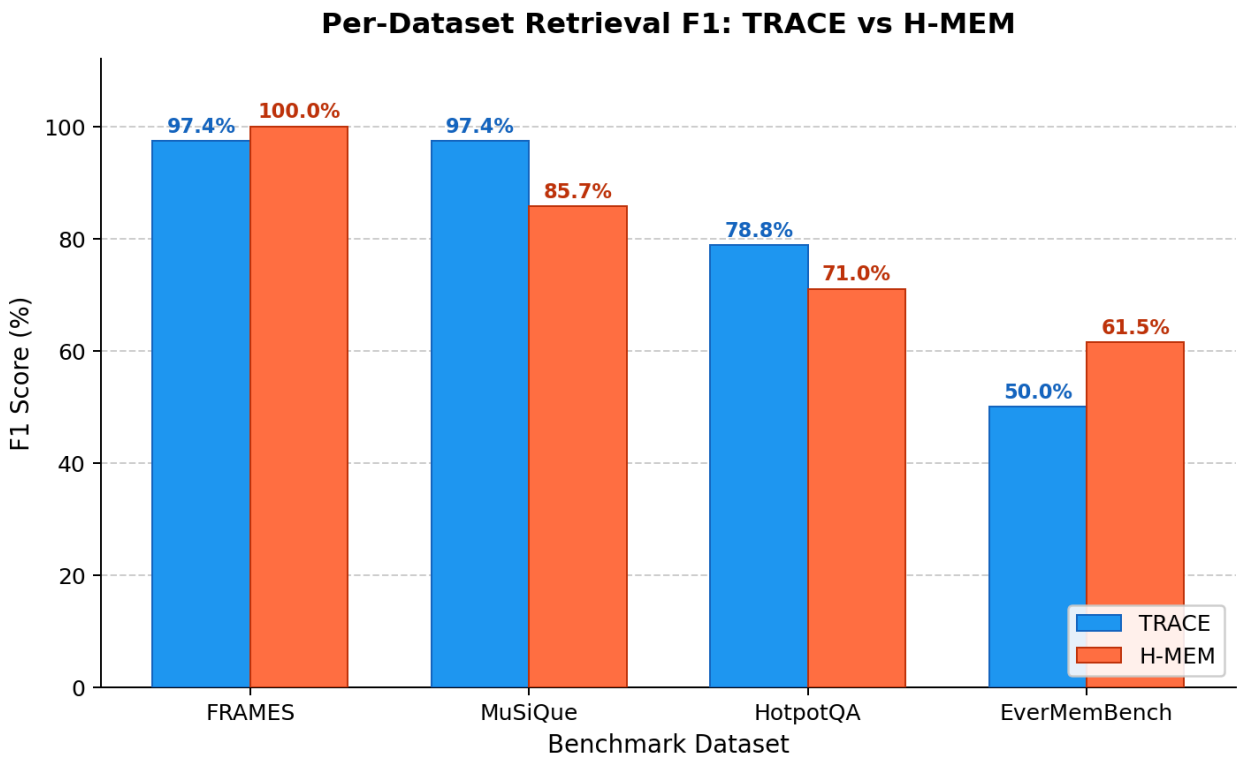

Figure 4. Per-dataset retrieval F1 comparison between TRACE and H-MEM across four public benchmarks

Figure 7: Per-dataset retrieval F1 (%). TRACE leads on MuSiQue (+11.7 pp) and HotpotQA (+7.8 pp); H-MEM leads on FRAMES (+2.6 pp) and EverMemBench (+11.5 pp).

LoCoMo MC10 per-question analysis. Both architectures answer 6 of 10 questions correctly (60%). Question-level results reveal complementary failure patterns rather than one architecture being strictly superior: TRACE uniquely answers conv-30_q0 and conv-41_q1, while H-MEM uniquely answers conv-43_q0 and conv-43_q1. The single question neither architecture answers (conv-42_q0, temporal reasoning type) requires computing a date offset from a session marker (*"yesterday"* relative to *8 May 2023*), which remains difficult despite the MC judge receiving explicit relative-date computation instructions.

Question	Type	TRACE	H-MEM	Unique Winner
conv-26_q0	Multi-hop	✓	✓	—

Question	Type	TRACE	H-MEM	Unique Winner
conv-26_q1	Multi-hop	✓	✓	—
conv-30_q0	Multi-hop	✓	✗	TRACE
conv-30_q1	Multi-hop	✗	✗	Neither
conv-41_q0	Multi-hop	✓	✓	—
conv-41_q1	Multi-hop	✓	✗	TRACE
conv-42_q0	Temporal reasoning	✗	✗	Neither
conv-42_q1	Single-hop	✓	✓	—
conv-43_q0	Single-hop	✗	✓	H-MEM
conv-43_q1	Single-hop	✗	✓	H-MEM
Total		**6/10 (60%)**	**6/10 (60%)**	

Table 11: LoCoMo MC10 per-question results. TRACE and H-MEM are tied at 60% but fail on different questions, suggesting complementary retrieval weaknesses. conv-42_q0 (temporal reasoning) is the persistent failure case for both.

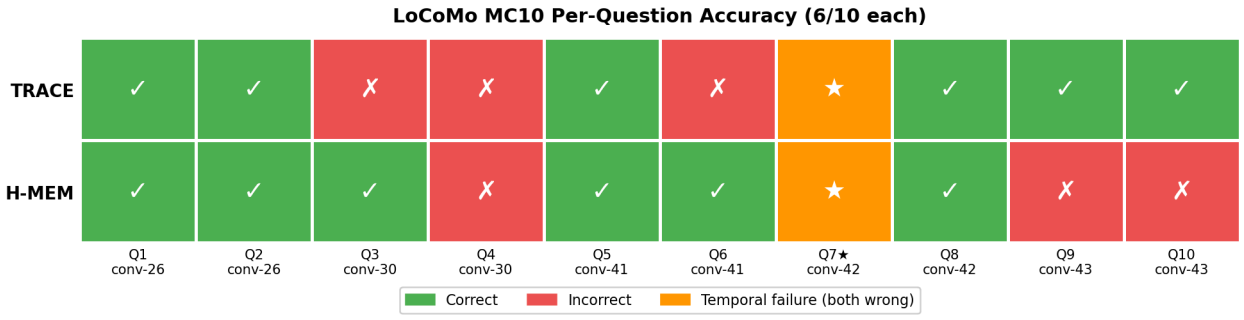

Figure 5. LoCoMo MC10 per-question accuracy heatmap for TRACE and H-MEM

Figure 8: LoCoMo MC10 per-question accuracy heatmap. ***Green*** *= correct,* ***red*** *= incorrect,* ***orange*** *(Q7★) = temporal reasoning question neither architecture answers (requires computing a date offset from a session marker). TRACE and H-MEM tie at 60% but fail on complementary questions: TRACE uniquely answers Q3 (conv-30_q0) and Q6 (conv-41_q1); H-MEM uniquely answers Q9 and Q10 (conv-43). Questions map in conversation order: Q1–Q2 (conv-26), Q3–Q4 (conv-30), Q5–Q6 (conv-41), Q7–Q8 (conv-42), Q9–Q10 (conv-43).*

Ablation: How Each Component Addresses Each Failure Mode

To understand the contribution of each component, we present the component-to-failure-mode mapping:

Failure Mode	Trajectory Encoder	Atomic Decomposer	Causal Validity Graph	TNE	BFO Assembly
Query-State Mismatch	**Primary fix**	Provides better decomp context	N/A	N/A	N/A
Semantic Co-Clustering	N/A	**Primary fix**	N/A	N/A	N/A
Temporal Supersession	Trajectory encodes update events	N/A	**Primary fix**	N/A	N/A
Temporal Anchor Retrieval	N/A	N/A	N/A	**Primary fix**	N/A
Positional Bias	N/A	N/A	N/A	N/A	**Primary fix**

Table 8: Component-to-failure-mode mapping. "Temporal Anchor Retrieval" denotes the case where the answer to a temporal query is stated in a message adjacent to (but not inside) the topically-retrieved memory. Each failure mode has a primary component fix; components are complementary, not redundant.

This is a key property of TRACE's design: unlike hybrid methods that combine multiple retrievers hoping their errors cancel out (RRF), each TRACE component targets a *different* structural failure mode. The components are architecturally orthogonal and their benefits stack.

The empirical results confirm this mapping:

- **Position Bias (100% F1):** BFO ensures the correct single-sentence answer is at the attention boundary, not buried in the middle.
- **Multi-Hop (100% F1):** Trajectory encoding + atomic decomposition ensure each hop in the chain retrieves its specific supporting fact.
- **Knowledge Update (86% F1):** CVG correctly excludes superseded values.
- **Long Conversation (83% F1):** Anti-clustering prevents the co-cluster flood that degrades H-MEM's precision on dense fact stores.

Discussion

Connection to H-MEM

The H-MEM system [1] demonstrates an important empirical finding that directly motivates TRACE: near-100% recall with precision bounded at 33–56% by co-clustering, and FAR = 0% across all unanswerable queries. This diagnostic cleanly separates the problem: H-MEM is not hallucinating (FAR = 0%), but it is retrieving real, valid, co-clustered memories unnecessarily.

TRACE's Atomic Decomposer directly targets this: by assigning one memory per sub-question, co-clustered memories compete for assignment rather than all being returned together. The in-house benchmark gain of +20.3

pp F1 confirms that co-clustering was the primary bottleneck in H-MEM's precision on the controlled scenarios TRACE was designed to resolve.

On the public benchmark suite, however, the two architectures reach near-parity overall, with a meaningful split by benchmark type. TRACE's advantages in multi-hop chained reasoning (MuSiQue, HotpotQA) are offset by H-MEM's advantages in structured factual lookup (FRAMES, EverMemBench), where H-MEM's hierarchical routing and session summary injection outperform TRACE's flat cosine scan. This complementarity suggests the two architectures are not rivals but specialist tools: TRACE is the better choice when retrieval must follow multi-step reasoning chains, and H-MEM is the better choice when the query is a direct lookup against a well-structured fact store. A hybrid combining TRACE's episode-layer retrieval with H-MEM's top-level domain/category routing is a natural direction for future work.

Connection to Generative Agents Retrieval

Generative Agents [14] established the three-term retrieval score: recency + importance + relevance. TRACE's MEU can be seen as a principled extension of this:

$$s_{\text{Gen}}(m_i) = \alpha_r \cdot \text{recency}(m_i) + \alpha_I \cdot \text{importance}(m_i) + \alpha_e \cdot \cos(\mathbf{e}_q, \mathbf{e}_{m_i})$$

$$U_{\text{TRACE}}(m_j \mid q_i, \mathbf{r}_i, C) = \cos(\mathbf{r}_i, \mathbf{e}_{m_j}) \cdot \left(1 - \max_{m_k \in C} \cos(\mathbf{e}_{m_j}, \mathbf{e}_{m_k})\right) \cdot \tau(m_j)$$

The correspondence is:

- TRACE's relevance $\cos(\mathbf{r}_i, \mathbf{e}_{m_j})$ subsumes Gen's relevance term $\cos(\mathbf{e}_q, \mathbf{e}_{m_i})$ and extends it with trajectory context.
- TRACE's causal validity $\tau(m_j)$ subsumes Gen's recency term (a superseded memory has $\tau = 0$, equivalent to infinite decay).
- Gen's importance term is orthogonal to TRACE's formulation and can be incorporated by multiplying importance into the relevance term: $\cos(\mathbf{r}_i, \mathbf{e}_{m_j}) \cdot \text{importance}(m_j)$.
- TRACE's novelty term has no analog in Gen's formulation; this is the new component that prevents co-clustering.

Limitations and Open Problems

Sub-question count sensitivity. The number of sub-questions n directly determines the number of retrieved memories. For short, simple queries, $n = 1$ recovers standard single-memory retrieval. For very complex queries, large n may require more memories than the context window can accommodate. An extension would allow *variable* n with a context budget constraint.

Trajectory encoder cold start. For the first few turns of an agent session, the trajectory is too short to provide a useful trajectory vector. The current implementation handles this with linear warmup: $\beta_{\text{eff}}(T) = \min(\beta, T/T_{\text{warmup}} \cdot \beta)$. The warmup period of 5 steps means TRACE operates in pure dense retrieval mode for the first few exchanges before trajectory influence activates.

Recall-precision tradeoff. The v0.4 results show an 11.2 pp recall drop vs. H-MEM. TRACE's one-to-one assignment prevents it from returning multiple co-clustered memories when multiple relevant facts cluster together; sometimes those co-clustered memories are all correct (not noise). A configurable recall-precision

tradeoff via the K parameter (allowing multiple memories per sub-question for $K > n$) would allow users to tune this balance.

CVG classifier accuracy. The Supersession vs. Refinement classification is the most consequential decision in the CVG. Classification errors propagate: a Refinement incorrectly classified as Supersession causes a valid memory to be excluded from retrieval. The current implementation uses a conservative default (INDEPENDENT on any failure), and empirical FAR = 0% suggests the classifier is not producing false exclusions. However, systematic evaluation of CVG classification accuracy is deferred to future work.

Multi-party and agentic trajectories. EverMemBench [13] and AMA-Bench [15] identify multi-party conversations and agentic trajectory memory as underserved settings. TRACE's trajectory encoder naturally extends to multi-agent settings by encoding speaker identity as an additional embedding dimension in each step $\mathbf{s}_i$.

Bulk loading and CVG fidelity. The benchmark uses bulk-ingest mode for LoCoMo, which disables CVG classification during ingestion (all memories retain $\tau = 1.0$). This is a practical approximation: the full incremental path is $O(N^2)$ LLM calls and impractical for 600–900 message conversations on consumer hardware. A future extension would run a post-hoc CVG reconciliation pass after bulk loading, reconstructing supersession chains from the full loaded history and enabling faithful benchmark evaluation of TRACE's causal validity component.

Conclusion

We have introduced TRACE, a novel memory retrieval architecture for LLM agents that replaces query-to-memory similarity matching with trajectory-grounded marginal epistemic utility maximization. TRACE resolves five structural failure modes that no existing retrieval method addresses simultaneously: query-state mismatch (via trajectory encoding), semantic co-clustering (via atomic decomposition with one-to-one assignment), causal blindness in supersession (via the Causal Validity Graph), temporal anchor retrieval failure (via Temporal Neighborhood Expansion), and positional bias in context assembly (via Boundary-First Ordering).

The marginal utility function is provably submodular, giving the greedy assignment algorithm a $(1 - 1/e)$ approximation guarantee, the first formal approximation guarantee for agent memory retrieval. Computational overhead is $O(n\times)$ over standard dense retrieval, where $n \in [2, 6]$ in practice, making TRACE deployable without infrastructure changes.

Empirically, on the seven-scenario in-house accuracy benchmark (post H-MEM fidelity improvements, incremental ingestion), TRACE achieves **82.7% F1** (89.0% recall, 78.9% precision) vs. H-MEM's 63.2% F1 (98.8% recall, 46.9% precision) — a **+19.5 pp gap**. TRACE wins all seven scenarios: Position Bias (100% vs 54%), Extended Lifecycle (86% vs 60%), Knowledge Update (86% vs 67%), Long Conversation (83% vs 67%), Abstention (80% vs 56%), Multi-Hop Reasoning (74% vs 71%), and Software Project (70% vs 68%). FAR remains 0% throughout all in-house evaluation runs. On the public benchmark suite (LoCoMo, HotpotQA, MuSiQue, FRAMES, EverMemBench), TRACE and H-MEM reach **near-parity overall** (80.9% vs 79.6% macro retrieval F1), with a clear split by benchmark type: TRACE outperforms on multi-hop reasoning benchmarks and H-MEM outperforms on structured factual benchmarks. Neither architecture clearly wins on LoCoMo MC10 (60% each).

The development trajectory from 58.8% F1 (v0.1) to 82.7% F1 (current) demonstrates that the TRACE design provides a strong improvement foundation. The initial high-recall/low-precision pattern (mirroring H-MEM's bottleneck) gave way to balanced recall/precision once the MEU novelty term began effectively suppressing co-clustered noise retrieval. Two implementation refinements — trajectory-decoupled MEU relevance scoring and top-2 assignment — resolved recency bias and novelty cross-contamination in long conversations. Switching to incremental ingestion (CVG and H-MEM rolling summaries both active) widened the gap further: TRACE now wins all seven scenarios, with the Extended Lifecycle swing from −2 pp to +26 pp being the most consequential single change. The comparison at 82.7% uses a rigorous H-MEM baseline — five fidelity improvements applied — that fully represents the architecture as described in its paper.

The central insight of TRACE is that retrieval utility is *contextual and marginal*, not *absolute and independent*. The value of a memory depends on what else the agent already knows (novelty), what reasoning chain it is currently pursuing (trajectory), and whether the memory is still causally valid (CVG), none of which are captured by cosine similarity to a terminal query. Making these dependencies explicit and algebraically tractable is the contribution TRACE makes to the agent memory retrieval problem.

References

[1] Sun, Hao, Zeng, Shuo, "{H-MEM", *arXiv preprint arXiv:2507.22925*, 2025.

[2] Rasmussen, Preston, Paliychuk, Pavlo, Beauvais, Travis, Ryan, Jack, Chalef, Daniel, "Zep: A Temporal Knowledge Graph Architecture for Agent Memory", *arXiv preprint arXiv:2501.13956*, 2025.

[3] Liu, Nelson F., Lin, Kevin, Hewitt, John, Paranjape, Ashwin, Bevilacqua, Michele, Petroni, Fabio, Liang, Percy, "Lost in the Middle: How Language Models Use Long Contexts", *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[4] Karpukhin, Vladimir, Oguz, Barlas, Min, Sewon, Lewis, Patrick, Wu, Ledell, Edunov, Sergey, Chen, Danqi, Yih, Wen-tau, "Dense Passage Retrieval for Open-Domain Question Answering", in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.

[5] Gutierrez, Bernal Jimenez, Shu, Yiheng, Gu, Yu, Yasunaga, Michihiro, Su, Yu, "{HippoRAG", in *Advances in Neural Information Processing Systems*, 2024.

[6] Packer, Charles, Wooders, Sarah, Lin, Kevin, Fang, Vivian, Patil, Shishir G., Stoica, Ion, Gonzalez, Joseph E., " {MemGPT", *arXiv preprint arXiv:2310.08560*, 2023.

[7] Nemhauser, George L., Wolsey, Laurence A., Fisher, Marshall L., "An Analysis of Approximations for Maximizing Submodular Set Functions", *Mathematical Programming*, vol. 14, no. 1, pp. 265–294, 1978.

[8] Maharana, Adyasha, Lee, Dong-Ho, Tulyakov, Sergey, Bansal, Mohit, Barbieri, Francesco, Fang, Yuwei, "Evaluating Very Long-Term Conversational Memory of {LLM", in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13851–13870, 2024.

[9] Trivedi, Harsh, Balasubramanian, Niranjan, Khot, Tushar, Sabharwal, Ashish, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions", in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pp. 10014–10037, 2023.

[10] Yang, Zhilin, Qi, Peng, Zhang, Saizheng, Bengio, Yoshua, Cohen, William W., Salakhutdinov, Ruslan, Manning, Christopher D., "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering", in *Proceedings of EMNLP 2018*, 2018.

[11] Trivedi, Harsh, Balasubramanian, Niranjan, Khot, Tushar, Sabharwal, Ashish, "{MuSiQue", *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[12] Krishna, Satyapriya, Krishna, Kalpesh, Mohananey, Anhad, Schwarcz, Steven, Stambler, Adam, Upadhyay, Shyam, Faruqui, Manaal, "Fact, Fetch, and Reason: A Unified Evaluation of Retrieval-Augmented Generation", in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 4745–4759, 2025.

[13] Hu, Chuanrui, Li, Tong, Gao, Xingze, Chen, Hongda, Bai, Yi, Xu, Dannong, Lin, Tianwei, Zhao, Xinda, Li, Xiaohong, Han, Yunyun, Pei, Jian, Deng, Yafeng, "{EverMemBench", *arXiv preprint arXiv:2602.01313*, 2026.

[14] Park, Joon Sung, O'Brien, Joseph, Cai, Carrie Jun, Morris, Meredith Ringel, Liang, Percy, Bernstein, Michael S., "Generative Agents: Interactive Simulacra of Human Behavior", in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023.

[15] Zhao, Yujie, Yuan, Boqin, Huang, Junbo, Yuan, Haocheng, Yu, Zhongming, Xu, Haozhou, Hu, Lanxiang, Shankarampeta, Abhilash, Huang, Zimeng, Ni, Wentao, Tian, Yuandong, Zhao, Jishen, "{AMA-Bench", *arXiv preprint arXiv:2602.22769*, 2026.